كلية هندسة الذكاء الاصطناعي
FACULTY OF ARTIFICIAL INTELLIGENCE ENGINEERING

الجامعة السورية الخاصة
SYRIAN PRIVATE UNIVERSITY

SignStream

Real-Time ASL Recognition & Translation System

Senior 2 Project

Student preparation:

Mohamad Barazi

Mohamad Khir Alhomsi

Supervised by:

Dr. Nisreen Sulayman

Eng. Zeinab Baghdadi

Damascus

Academic Year 2025 - 2026

SUPERVISOR CERTIFICATION

I certify that the preparation of this project, entitled

"**SignStream Real-Time ASL Recognition & Translation System"**

Prepared by

"**Mohammad Barazi, Mohammad Khir Alhomsi"**

Was carried out under me supervision at the faculty of Artificial Intelligence Engineering in partial fulfillment of the requirements for the degree of **B.Sc.** in the Department of Artificial Intelligence Engineering and Data Science.

Name:

Date:

Signature:

Acknowledgements

This senior project was conducted at the Faculty of Artificial Intelligence Engineering, Syrian Private University. We would like to express our sincere appreciation to our supervisors, Dr. Nisreen Sulayman and Eng. Zeinab Baghdadi, for their academic guidance, technical feedback, and continuous support throughout the development of this work. Their observations helped us refine the system objective, improve the experimental design, and organize the project into a complete recognition and translation pipeline.

We also thank the teaching staff of the Department of AI & Data Science for providing the theoretical and practical foundation required to build deep learning systems, computer vision pipelines, and evaluation workflows. Finally, we acknowledge the open research communities that released datasets, libraries, and reproducible tools used in this project, including ASL Citizen, MediaPipe, PyTorch, Hugging Face Transformers, and the sign language research literature.

Dedication

We dedicate this work to our families, whose support made the completion of this project possible. We also dedicate it to students and researchers working on assistive artificial intelligence systems, especially those focused on improving accessibility for Deaf and hard-of-hearing communities. The project is a small contribution toward more inclusive human-computer interaction and more accessible communication technology.

الملخص

يُعدّ التعرّف على لغة الإشارة مشكلةً مهمةً في مجال رؤية الحاسوب والذكاء الاصطناعي، إذ يُسهم في تسهيل التواصل للصم وضعاف السمع. لغة الإشارة الأمريكية لغةٌ بصريةٌ تعتمد على حركة اليد وشكلها، ووضعية الجسم، وتعبيرات الوجه، وأنماط الحركة الزمنية. وهذا ما يجعل التعرّف التلقائي عليها تحديًا، لا سيما عند التعامل مع اختلاف المُشيرين، وتشابه الإشارات بصريًا، والحركة السريعة، والحجب، والضوضاء الخلفية، والحاجة إلى استجابة فورية. إضافةً إلى ذلك، يتطلب تحويل تنبؤات الإشارات المنفردة إلى نصٍّ وكلامٍ ذي معنى تكاملًا بين التعرّف البصري، ومعالجة اللغة الطبيعية، وتوليف الكلام.

يقترح هذا المشروع SignStream، وهو نظامٌ للتعرّف على لغة الإشارة الأمريكية وترجمتها في الوقت الفعلي. يتبع النظام مسارًا متكاملًا: إدخال الفيديو ← استخراج الميزات ← التنبؤ بالنموذج ← توليد النص ← تحويل النص إلى كلام. في البداية، تُعالَج إطارات الفيديو باستخدام MediaPipe لاستخراج النقاط الرئيسية من اليدين، ووضعية الجزء العلوي من الجسم، ومناطق مُختارة من الوجه كالشفتين والعينين والحاجبين. بعد ذلك، تُمرر تسلسلات النقاط الرئيسية المستخرجة إلى نموذج تعلم عميق زمني يعتمد على طبقات التفافية ومُشفِّرات Transformer لتصنيف دلالات لغة الإشارة الأمريكية. يتضمن النظام أيضًا واجهة مستخدم تدعم كلاً من إدخال كاميرا الويب المباشر والفيديو المُحمَّل. بعد التعرف، يمكن تحويل الدلالات المتوقعة إلى نص مقروء ومخرجات صوتية باستخدام وحدة تحويل النص إلى كلام. تكمن قيمة النظام المقترح في دمج التعرف البصري، والنمذجة الزمنية، وتصميم الواجهة، والإخراج الصوتي في نموذج أولي عملي واحد.

تم تدريب نموذج V3-Large المقترح وتقييمه على مجموعة بيانات ASL Citizen باستخدام 2731 فئة دلالات. حقق النموذج أفضل دقة تحقق من الدرجة الأولى (Top-1) بنسبة 74.70% ودقة تحقق من الدرجة الخامسة (Top-5) بنسبة 93.23%. على مجموعة الاختبار، حقق النموذج دقة تحقق من الدرجة الأولى (Top-1) بنسبة 67.40% ودقة تحقق من الدرجة الخامسة (Top-5) بنسبة 89.31% باستخدام تقييم EMA. بالمقارنة مع النموذج الأساسي السابق، الذي حقق دقة اختبار Top-1 بنسبة 58.56%، حسّن النموذج المقترح الأداء بنسبة 8.84 نقطة مئوية. تُظهر هذه النتائج فعالية النمذجة الزمنية القائمة على النقاط الرئيسية في التعرف على لغة الإشارة الأمريكية ذات المفردات الكبيرة، مع الحفاظ على ملاءمتها للدمج في نموذج أولي للترجمة الفورية. ومع ذلك، لا يزال تباين المُشيرين، والتشابه البصري في الشروح، والتعرف المستمر على مستوى الجملة، تحديات مهمة للتطوير المستقبلي.

Abstract

Sign language recognition is an important computer vision and artificial intelligence problem because it supports communication accessibility for deaf and hard-of-hearing individuals. American Sign Language is a visual language that depends on hand motion, hand shape, body posture, facial expressions, and temporal movement patterns. This makes automatic recognition challenging, especially when dealing with signer variation, visually similar signs, fast motion, occlusion, background noise, and the need for real-time response. In addition, translating isolated sign predictions into meaningful text and speech requires integration between visual recognition, natural language processing, and speech synthesis.

This project proposes **SignStream**, a real-time ASL recognition and translation system. The system follows a complete pipeline: **Video Input → Feature Extraction → Model Prediction → Text Generation → Text-to-Speech**. First, video frames are processed using MediaPipe to extract keypoints from the hands, upper-body pose, and selected facial regions such as lips, eyes, and eyebrows. Then, the extracted keypoint sequences are passed to a temporal deep learning model based on convolutional layers and Transformer encoders to classify ASL glosses. The system also includes a user interface that supports both live webcam input and uploaded video. After recognition, the predicted glosses can be converted into readable text and spoken output using a text-to-speech module. The value of the proposed system lies in combining visual recognition, temporal modeling, interface design, and speech output into one practical assistive prototype.

The proposed V3-Large model was trained and evaluated on the ASL Citizen dataset using 2,731 gloss classes. The model achieved a best validation Top-1 accuracy of **74.70%** and validation Top-5 accuracy of **93.23%**. On the test set, the model achieved **67.40% Top-1 accuracy** and **89.31% Top-5 accuracy** using EMA evaluation. Compared with the previous baseline model, which achieved **58.56%** test Top-1 accuracy, the proposed model improved performance by **8.84 percentage points**. These results show that keypoint-based temporal modeling is effective for large-vocabulary ASL recognition, while remaining suitable for integration into a real-time translation prototype. However, signer variability, visually similar glosses, and continuous sentence-level recognition remain important challenges for future development.

Keywords: American Sign Language, Computer Vision, Keypoint Extraction, Temporal Transformer, Text-to-Speech.

Table of Contents

List of Tables

List of Figures

List of Proposed Equations

List of Abbreviations and Symbols

Abbreviation	Meaning
AI	Artificial Intelligence
ASL	American Sign Language
BLEU	Bilingual Evaluation Understudy
BLEU-4	BLEU score using up to 4-gram matching
BLEURT	Bilingual Evaluation Understudy with Representations from Transformers
CNN	Convolutional Neural Network
CSL	Chinese Sign Language
CSL-Daily	Chinese Sign Language Daily dataset
CSLR	Continuous Sign Language Recognition
CSV	Comma-Separated Values
EMA	Exponential Moving Average
FPS	Frames per Second
GELU	Gaussian Error Linear Unit
GPU	Graphics Processing Unit
GUI	Graphical User Interface
ISLR	Isolated Sign Language Recognition
LLM	Large Language Model
mBART	Multilingual Bidirectional and Auto-Regressive Transformers
MSASL	Microsoft American Sign Language dataset
NLA-SLR	Natural Language-Assisted Sign Language Recognition
NLP	Natural Language Processing
NMFs-CSL	Chinese Sign Language recognition benchmark used in related work
NPZ	Compressed NumPy Array File
OpenCV	Open Source Computer Vision Library
PHOENIX14	RWTH-PHOENIX-Weather 2014 continuous sign language dataset
PHOENIX14-T / PHOENIX-2014T	RWTH-PHOENIX-Weather 2014T sign language translation dataset
PyQt5	Python Qt graphical interface library
RGB	Red, Green, Blue
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
SLR	Sign Language Recognition
SLT	Sign Language Translation
T5	Text-to-Text Transfer Transformer
TTS	Text-to-Speech
UI	User Interface
V3-Large	Proposed large SignTransformer model version
WER	Word Error Rate
WLASL	Word-Level American Sign Language dataset
Top-1	Accuracy when the best-ranked prediction equals the true label
Top-5	Accuracy when the true label appears among the five highest-ranked predictions
Recall@10	Metric measuring whether the correct sign appears among the top ten retrieved candidates

Chapter 1

Introduction

1.1 Introduction

Communication is a fundamental part of human interaction, education, social participation, and access to daily services. For deaf and hard-of-hearing individuals, sign language represents one of the main forms of communication. Unlike spoken languages, sign languages depend mainly on visual information, including hand movements, hand shapes, body posture, facial expressions, and the spatial relationship between different parts of the body. This makes sign language expressive and rich, but also difficult to automatically recognize using computer systems.

American Sign Language, commonly known as ASL, is a natural language with its own vocabulary and grammatical structure. It is not simply a direct translation of English words into hand gestures. Therefore, building an intelligent system for ASL recognition requires understanding visual motion over time and identifying meaningful patterns from video input. Such a system can help reduce the communication gap between sign language users and people who do not understand sign language.

In recent years, artificial intelligence, computer vision, and deep learning have become powerful tools for analyzing human motion from images and videos. These technologies can be used to detect human body landmarks, track hand movements, and classify temporal patterns. Instead of relying only on raw video frames, many modern systems extract important body and hand features, then use deep learning models to recognize the performed sign.

This project, titled **SignStream: Real-Time ASL Recognition and Translation System**, aims to develop an intelligent prototype that recognizes ASL signs from video and converts them into text and speech. The system supports both live camera input and uploaded video input. It follows a complete pipeline starting from video acquisition, then feature extraction, model prediction, text generation, and finally text-to-speech output.

The general pipeline of the proposed system is:

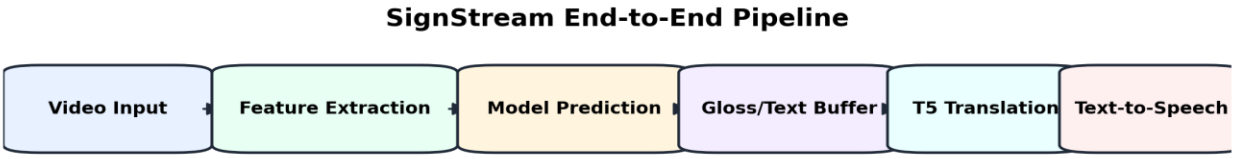

Figure 1signStream Pipeline

The project is designed as an assistive artificial intelligence system that combines computer vision, temporal deep learning, natural language processing, and speech synthesis in one practical application.

1.2 Research Problem

The scientific problem addressed in this project is the automatic recognition and translation of American Sign Language signs from video input. Although sign language is a natural and effective communication method for deaf and hard-of-hearing individuals, most people who do not learn sign language cannot understand it. This creates a communication barrier in education, public services, healthcare, workplaces, and daily social interactions.

Automatic sign language recognition is a challenging problem for several reasons. First, signs are dynamic actions that change over time, so the system must understand motion, not only static hand positions. Second, many ASL signs are visually similar, which makes classification difficult. Third, different signers may perform the same sign with variations in speed, hand position, body posture, and facial expression. Fourth, real-world video input may contain noise, different lighting conditions, partial occlusion, and background variation.

Another challenge is that recognizing isolated signs alone is not enough for practical communication. A complete assistive system should also convert the recognized signs into readable text and spoken output. Therefore, the research problem is not only a visual classification problem, but also an integration problem that requires combining visual recognition, temporal modeling, user interaction, and speech output.

This project addresses the following main research question:

How can an AI-based system recognize ASL signs from video input and convert them into text and speech in a practical and user-friendly way?

1.3 Research Objectives

The main objective of this project is to build an intelligent system capable of recognizing American Sign Language signs from video input and converting the recognized output into text and speech.

The specific objectives of the project are:

1. To design a complete ASL recognition pipeline that starts from video input and ends with text and speech output.
2. To extract meaningful visual features from video frames using keypoints from the hands, body pose, and selected facial regions.
3. To train a deep learning model capable of learning temporal patterns from keypoint sequences.
4. To classify ASL glosses using a large-vocabulary dataset containing thousands of sign classes.
5. To improve recognition performance compared with an earlier baseline model.
6. To provide a user interface that supports both live webcam input and uploaded video input.
7. To display prediction results, confidence scores, and top candidate signs in a clear way for the user.
8. To convert recognized signs into readable text and support speech output using a text-to-speech module.

9. To evaluate the system using suitable metrics such as Top-1 accuracy and Top-5 accuracy.
10. To develop a practical prototype that can serve as a foundation for future real-time sign language translation systems.

1.4 Research Contribution

The main contribution of this project is the development of an integrated ASL recognition and translation prototype that combines multiple artificial intelligence components into one complete system.
The proposed system contributes in the following ways:

1. **Integrated Pipeline:**
   The project combines video processing, feature extraction, temporal deep learning, text generation, and text-to-speech output in one unified workflow.
2. **Keypoint-Based Visual Representation:**
   Instead of processing raw video frames directly, the system uses extracted keypoints from the hands, pose, and selected facial regions. This reduces the influence of background noise and focuses the model on human motion.
3. **Temporal Deep Learning Model:**
   The project uses a deep temporal model that combines convolutional layers and Transformer-based sequence modeling to recognize motion patterns across time.
4. **Large-Vocabulary ASL Recognition:**
   The model is trained on a large set of ASL gloss classes, making the task more realistic and more challenging than small gesture classification problems.
5. **Real-Time-Oriented Interface:**
   The system includes a desktop interface that supports live camera mode, uploaded video mode, prediction visualization, and session history.
6. **Speech Output Support:**
   The recognized text can be converted into speech, making the system closer to a practical communication-support tool.
7. **Performance Improvement over Baseline:**
   The proposed V3-Large model achieved better test performance than the previous baseline model, showing the effectiveness of the improved architecture and training strategy.

1.5 Beneficiaries of the Research

The proposed system can benefit several groups and sectors:

1. **Deaf and Hard-of-Hearing Individuals:**
   The system can support communication by converting ASL signs into text and speech.
2. **People Who Do Not Know Sign Language:**
   The system can help non-signers understand basic ASL communication.

3. **Educational Institutions:**
   Schools, universities, and training centers can use the project as a prototype for assistive educational technologies.
4. **Healthcare and Public Service Centers:**
   The system can help improve accessibility in environments where communication with deaf or hard-of-hearing individuals is important.
5. **Researchers and Students:**
   The project provides a practical example of combining computer vision, deep learning, natural language processing, and human-computer interaction.
6. **Assistive Technology Developers:**
   The system can serve as a base for future applications related to real-time sign language recognition and translation.

1.6 Thesis Organization

This thesis is organized into six chapters.

Chapter 1 introduces the general background of the project, defines the research problem, presents the research objectives, explains the contribution of the project, identifies the potential beneficiaries, and describes the structure of the thesis.

Chapter 2 presents the theoretical background required to understand the project. It discusses sign language recognition, computer vision, keypoint extraction, deep learning, temporal sequence modeling, Transformer models, natural language processing, and text-to-speech systems.

Chapter 3 reviews related works and previous studies in sign language recognition and translation. It compares existing approaches, discusses their strengths and limitations, and identifies the research gap addressed by this project.

Chapter 4 explains the proposed methodology. It describes the dataset, preprocessing steps, feature extraction process, model architecture, training strategy, system pipeline, and user interface design.

Chapter 5 presents the experimental results and evaluation. It discusses the evaluation metrics, model performance, comparison with the baseline model, per-signer analysis, and system behavior.

Chapter 6 concludes the thesis by summarizing the main findings, discussing the limitations of the project, and presenting possible future improvements.

Chapter 2

Theoretical Study

2.1 Introduction

This chapter presents the theoretical background required to understand the proposed SignStream system. The project is based on several connected fields, including sign language recognition, computer vision, keypoint extraction, deep learning, temporal sequence modeling, natural language processing, and text-to-speech synthesis. The purpose of this chapter is to explain the main concepts and computational foundations used in the system before presenting the proposed methodology in the following chapters.

The proposed system does not rely only on a single artificial intelligence technique. Instead, it combines multiple components: visual feature extraction from video, temporal modeling of motion, classification of ASL glosses, text generation, and speech output. Therefore, understanding the theoretical basis of each component is necessary to understand how the complete system works.

2.2 American Sign Language Recognition

American Sign Language, or ASL, is a visual language used by many deaf and hard-of-hearing individuals. It uses hand shapes, hand movements, facial expressions, body posture, and spatial relationships to represent meaning. Unlike spoken language, which is mainly audio-based, ASL is processed visually and temporally.

Automatic ASL recognition is the task of using a computer system to identify signs from image or video input. In this project, the system receives video frames and predicts the corresponding ASL gloss. A gloss is a written label that represents the meaning of a sign. For example, a sign performed in the video may be represented by a gloss such as **BOOK**, **HELP**, or **THANK-YOU**.

ASL recognition is difficult because signs are dynamic. The meaning of a sign may depend not only on the final hand position, but also on the movement path, speed, hand orientation, body location, and facial expression. Therefore, sign language recognition is considered a temporal visual recognition problem.

2.3 Computer Vision for Human Motion Understanding

Computer vision is a branch of artificial intelligence that enables computers to analyze and understand visual information from images and videos. In the context of this project, computer vision is used to process video frames and extract information related to the signer's body, hands, and face.

A video can be represented as a sequence of frames:

$V = \{I_1, I_2, I_3, \ldots, I_T\}$where V represents the video, It represents the frame at time t, and T is the total number of frames.

The objective is to transform this sequence of raw frames into a structured representation that can be used by a machine learning model. Processing raw video frames directly is computationally expensive because each frame contains many pixels, including background information that may

not be useful for sign recognition. Therefore, many sign recognition systems extract compact visual features such as human keypoints.

2.4 Keypoint-Based Representation

A keypoint is a specific landmark point detected on the human body, hands, or face. For sign language recognition, keypoints are very useful because they describe the motion of the signer without requiring the model to process the entire image.
In this project, keypoints are extracted from three main regions:

1. **Hands:** because hand shape and hand motion are central to sign language.
2. **Upper-body pose:** because signs may depend on body position and arm movement.
3. **Selected facial regions:** such as lips, eyes, and eyebrows, because facial expressions can support meaning in sign language.

Each keypoint can be represented by coordinates such as:

$$pi = (xi, yi, zi)$$

where ***xi, yi,*** and ***zi*** represent the spatial position of keypoint ***i***. For a full frame, all extracted keypoints are combined into one feature vector:

$$F_t = [p_1, p_2, p_3, \ldots, p_n]$$

where ***Ft*** is the feature vector at frame ***t***, and ***n*** is the number of extracted keypoints.
For a complete video sequence, the extracted features can be represented as:

$$X = \{F_1, F_2, F_3, \ldots., F_T\}$$

This means that the video is converted into a temporal sequence of keypoint features.

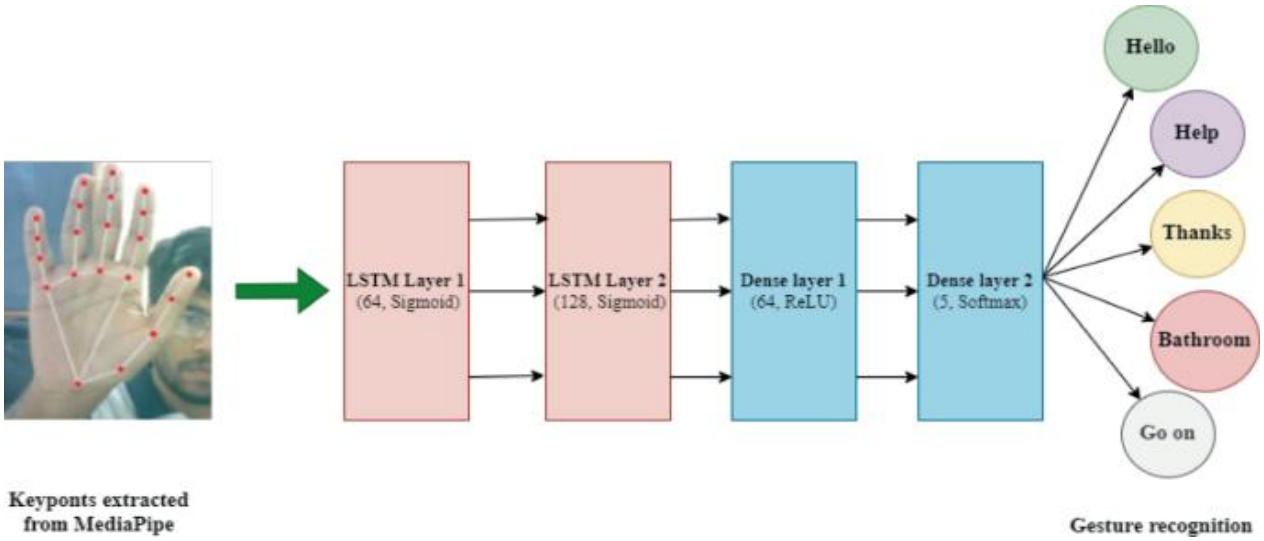

Figure 2 mediapipe architecture

2.5 Feature Normalization

Feature normalization is an important preprocessing step in sign language recognition. Different signers may appear at different positions in the camera frame, may have different body sizes, and may perform signs at different distances from the camera. Without normalization, the model may learn irrelevant differences instead of learning the actual sign motion.
A common normalization method is to center the keypoints around a reference body point and scale them based on body size. For example, if c represents a body center point and s represents a body scale factor, a normalized keypoint can be computed as:

$$p'_i = \frac{p_{i-c}}{s}$$

where p_i is the original keypoint and $\boldsymbol{pi'}$ is the normalized keypoint.

This transformation reduces the effect of signer position and distance from the camera. As a result, the model can focus more on the relative motion and shape of the sign.

2.6 Temporal Motion Features

Since sign language is based on movement, static keypoints alone are not always sufficient. The system must also understand how keypoints change over time. For this reason, temporal features such as velocity and acceleration can be used.
The velocity of a keypoint sequence measures how the position changes between consecutive frames:

$$v_t = p_t - p_{t-1}$$

The acceleration measures how the velocity changes over time:

$$a_t = v_t - v_{t-1}$$

Velocity helps the model understand movement direction and speed, while acceleration helps capture sudden changes in motion. These temporal features are useful for distinguishing signs that may have similar hand positions but different movement patterns.

2.7 Deep Learning for Sign Classification

Deep learning is a subfield of machine learning that uses neural networks with multiple layers to learn patterns from data. In sign language recognition, deep learning models can learn the relationship between visual motion features and sign labels.
The input to the model is a sequence of extracted features:

$$X \in R^{T \times F}$$

where ***T*** is the number of frames and ***F*** is the feature dimension per frame.
The output is a probability distribution over the possible sign classes:

$$\hat{y} = [P(c_1), P(c_2), \ldots, P(c_k)]$$

where ***k*** is the number of sign classes, and $P(c_i)$is the predicted probability of class ***i***.
The predicted class is selected as:

$$\hat{c} = argmax P(c_i)$$

This means that the model chooses the class with the highest predicted probability.

2.8 Convolutional Neural Networks for Temporal Patterns

Convolutional Neural Networks, or CNNs, are commonly used to detect local patterns in data. Although CNNs are widely known for image processing, one-dimensional convolution can also be used for temporal sequences.
In this project domain, temporal convolution can detect short-term movement patterns across consecutive frames. A 1D convolution operation can be represented as:

$$y_t = \sum_{i=-r}^{r} w_i x_{t+i} + b$$

where x_{t+i} represents neighboring input frames, w_i represents learnable weights, ***b*** is a bias term, and y_t is the output feature at time ***t***.
Temporal CNN layers are useful because they can capture local motion patterns such as hand movement direction, short pauses, and transitions between sign phases. They also reduce noise by learning stable motion representations from neighboring frames.

2.9 Transformer Models and Self-Attention

Transformer models are deep learning architectures designed to process sequential data. Unlike recurrent models, Transformers use an attention mechanism that allows the model to focus on important parts of the input sequence.
The main component of a Transformer is the self-attention mechanism. Self-attention allows each frame in the sequence to compare itself with other frames and determine which frames are most relevant.
Given an input sequence X, the model computes three matrices:

$$\boldsymbol{Q} = \boldsymbol{X}W_Q$$

$$\boldsymbol{K} = \boldsymbol{X}W_K$$

$$\boldsymbol{V} = \boldsymbol{X}W_V$$

where $\boldsymbol{Q}$ is the query matrix, $\boldsymbol{K}$ is the key matrix, $\boldsymbol{V}$ is the value matrix, and W_Q , W_K, and W_V are learnable weight matrices.
The attention output is computed as:

$$\boldsymbol{Attention(Q , K, V)=softmax((}\frac{QK^T}{\sqrt{d_k}}\boldsymbol{)V}$$

where d_k is the dimension of the key vectors.
In sign language recognition, self-attention is useful because important frames may appear at different positions in the video. Some frames may contain the most meaningful hand shape, while others may contain the most important motion transition. The attention mechanism helps the model focus on these important temporal regions.

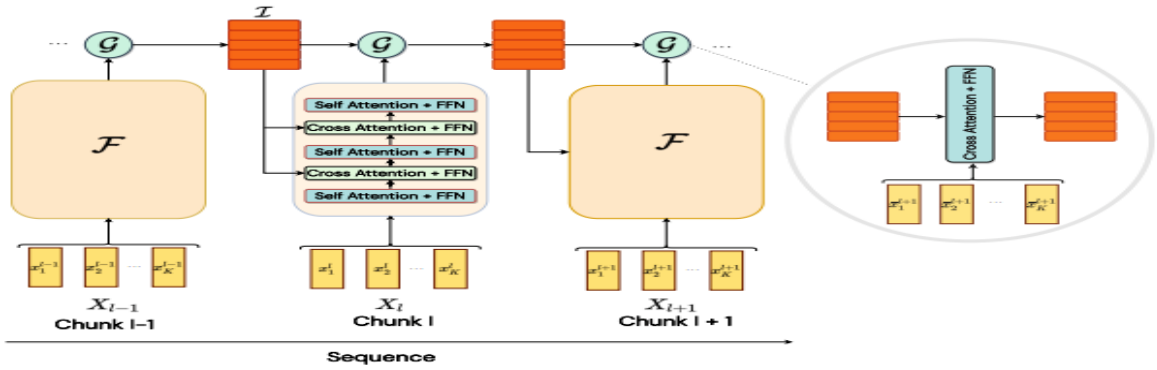

Figure 3 self-attention

2.10 Positional Encoding

Since the Transformer architecture does not process frames in a strictly sequential manner by default, it needs positional information to understand the order of frames. Positional encoding is added to the input features to represent the position of each frame in the sequence.
A common sinusoidal positional encoding is defined as:

$$\boldsymbol{PE\ (pos,\ 2i) = sin\ (\frac{pos}{10000^{2i/d}})}$$

$$\boldsymbol{PE\ (pos,\ 2i+1) = cos\ (\frac{pos}{10000^{2i/d}})}$$

where ***pos*** is the position of the frame, ***i*** is the feature dimension index, and ***d*** is the model dimension . This encoding allows the model to distinguish between early, middle, and late frames in a sign sequence. This is important because changing the order of movements can change the meaning of a sign.

2.11 Sequence Pooling

After the model processes the full sequence, the frame-level features must be converted into a single representation for classification. This is done using pooling.
Mean pooling computes the average representation across time:

$$h_{mean} = \frac{1}{T}\sum_{t=1}^{T} h_t$$

Max pooling selects the strongest activation across time:

$$h_{max} = \max_{t}(h_t)$$

Mean pooling captures the general information across the whole sign, while max pooling captures the most distinctive features. Combining both can improve classification because the model benefits from both global and strong local temporal information.

2.12 Softmax Classification

The final layer of a sign classification model produces logits, which are raw numerical scores for each class. These logits are converted into probabilities using the Softmax function:

$$\boldsymbol{P\ (c_i) = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}}}$$

where z_i is the logit of class ***i***, and ***k*** is the number of classes.
The Softmax function ensures that all output probabilities sum to one. The class with the highest probability is selected as the predicted sign.

2.13 Cross-Entropy Loss

For multi-class classification, Cross-Entropy Loss is commonly used to train the model. It measures the difference between the true class label and the predicted probability distribution. The Cross-Entropy Loss is defined as:

$$\boldsymbol{L} = -\sum_{i=1}^{k} y_i log\ (\hat{y}_i)$$

where y_i is the true label distribution and $\hat{y}_i$ is the predicted probability for class $\boldsymbol{i}$.
If the model gives a high probability to the correct class, the loss becomes small. If the model gives a low probability to the correct class, the loss becomes large. During training, the model updates its weights to minimize this loss.

2.14 Natural Language Processing in Sign Translation

Natural Language Processing, or NLP, is the field of artificial intelligence that focuses on processing and generating human language. In sign language systems, NLP can be used after visual recognition to convert predicted glosses into more natural text.
A gloss sequence may not always be grammatically identical to a spoken or written sentence. Therefore, an NLP module can help refine the recognized output, correct word order, and produce a more readable sentence. In the context of this project, NLP is considered an additional stage after visual recognition.
For example, the visual model may output a sequence of recognized glosses. The NLP component can then transform this sequence into a clearer textual sentence. This step improves the usability of the system because users usually need readable text rather than only isolated gloss labels.

2.15 Text-to-Speech Synthesis

Text-to-Speech, or TTS, is the process of converting written text into spoken audio. In assistive communication systems, TTS allows recognized signs to be delivered as audible speech.
A TTS module generally receives text as input and generates a corresponding speech signal as output:

$$\boldsymbol{Text} \rightarrow Speach$$

In this project, TTS is important because it makes the system more useful in real communication scenarios. Instead of displaying only text, the system can produce speech output, allowing communication with people who may not be looking at the screen or who prefer audio interaction.

2.16 Chapter Summary

This chapter presented the theoretical background of the SignStream project. It introduced American Sign Language recognition as a visual and temporal artificial intelligence problem. It explained the role of computer vision in extracting useful information from video frames and described keypoint-based representation as an efficient alternative to raw pixel processing.
The chapter also discussed feature normalization, temporal motion features, deep learning, temporal convolution, Transformer-based self-attention, positional encoding, pooling, Softmax classification, and Cross-Entropy Loss. Finally, it introduced the role of natural language processing and text-to-speech synthesis in converting recognized signs into readable and spoken output.
These theoretical concepts form the foundation of the proposed methodology, which will be described in detail in the following chapters.

Chapter 3

Related Work

3.1 Chapter Introduction

This chapter reviews recent research related to **SignStream: Real-Time ASL Recognition and Translation System**. The selected studies cover the main areas of the project, including isolated sign language recognition, ASL datasets, keypoint-based representation, temporal deep learning, continuous recognition, sign language translation, NLP, and large language models.
These studies show how modern AI methods such as keypoint extraction, Transformers, temporal modeling, and language-based supervision have been used to improve sign language recognition and translation. The chapter also helps identify the research gap addressed by SignStream, which integrates visual recognition, gloss buffering, NLP-based text generation, Text-to-Speech, and a user interface into one practical assistive prototype.

3.2 Previous Studies

3.2.1 Study 1:

ASL Citizen: A Community-Sourced Dataset for Advancing Isolated Sign Language Recognition

Research group and year:
Desai et al., 2023.

Proposed Work:
The authors introduced **ASL Citizen**, a large-scale crowdsourced dataset for isolated American Sign Language recognition. The dataset was created to support ASL dictionary retrieval, where a user signs in front of a webcam and the system retrieves the closest matching sign from a dictionary. This study is especially important for this project because SignStream uses the ASL Citizen dataset as the main dataset for training and evaluation. The released dataset contains **83,399 videos**, **2,731 distinct signs**, and **52 signers** collected in different environments.

Dataset:
The study used the ASL Citizen dataset, which contains isolated ASL video clips collected from multiple signers. The dataset was designed for signer-independent evaluation, meaning that test signers are not included in the training or validation sets.

Problem:
The main problem addressed in this study is the lack of large, diverse, and community-sourced datasets for American Sign Language recognition. Many earlier ASL datasets were limited in size, number of signs, number of signers, or environmental diversity.

Preprocessing:
The paper focused mainly on dataset construction and benchmark creation. The videos were organized into standard training, validation, and testing splits to support supervised machine learning evaluation.

Model used:
The authors trained supervised machine learning classifiers to establish baseline results for dictionary retrieval and isolated sign recognition.

Evaluation metrics:
The evaluation used accuracy and Recall@10, which are suitable for dictionary retrieval because the correct sign may appear among the top retrieved candidates.

Results:
The ASL Citizen baseline achieved **63% accuracy** and **91% Recall@10** on signer-independent evaluation, where the test users were not included in the training or validation sets. These results show that the dataset is challenging because the model must generalize to unseen signers while recognizing a large vocabulary of **2,731 ASL signs**.

Contribution:
The main contribution is the release of a large community-sourced ASL dataset and baseline results for isolated ASL recognition and dictionary retrieval.

3.2.2 Study 2:

Natural Language-Assisted Sign Language Recognition

Research group and year:
Zuo, Wei, and Mak, 2023.

Proposed Work:
The authors proposed a **Natural Language-Assisted Sign Language Recognition** framework. The main idea is to use semantic information from gloss labels to improve visual sign recognition, especially for visually indistinguishable signs. The paper introduced language-aware label smoothing, inter-modality mixup, and a video-keypoint network that models both RGB videos and human body keypoints.

Dataset:
The method was evaluated on commonly used sign language recognition benchmarks, including **MSASL**, **WLASL**, and **NMFs-CSL**.

Problem:
The study addressed the problem of visually similar signs. In sign language, different glosses may have very similar hand shapes, movements, or body positions. A purely visual model may confuse these signs because the visual differences are small.

Preprocessing:
The study used visual sign representations and keypoint information. It also incorporated semantic information from gloss labels to support the training process.

Model used:
The proposed framework included a video-keypoint network and language-assisted training strategies. Language-aware label smoothing was used for signs with similar semantic meanings, while inter-modality mixup was used to improve separability between signs with different meanings.

Evaluation metrics:
The method was evaluated using recognition accuracy on standard isolated sign language recognition datasets.

Results:
The proposed NLA-SLR method achieved strong results on several isolated sign language recognition benchmarks. On **WLASL2000**, the method achieved **61.26% Top-1** and **91.77% Top-5** accuracy using 3-crop inference. On **NMFs-CSL**, it achieved **83.7% Top-1** and **98.5% Top-5** accuracy using 3-crop inference. The paper also reported improvements over previous best methods on MSASL, WLASL, and NMFs-CSL.

Contribution:
The main contribution is introducing natural language supervision into sign language recognition to improve recognition of visually similar signs.

3.2.3 Study 3:

Preprocessing MediaPipe Keypoints with Keypoint Reconstruction and Anchors for Isolated Sign Language Recognition

Research group and year:
Roh et al., 2024.

Proposed Work:
The authors proposed preprocessing methods for MediaPipe keypoints in isolated sign language recognition. Their approach used keypoint reconstruction and anchor-based normalization to improve the quality and stability of keypoint representations. This study is highly relevant to SignStream because the proposed system also depends on MediaPipe keypoints for hands, pose, and selected facial regions.

Dataset:
The study evaluated its method mainly on the **WLASL-100** dataset.

Problem:
The paper addressed the problem of noisy or missing keypoints. In sign language recognition, hand keypoints may be missing or inaccurate due to fast motion, occlusion, detection failure, or poor video quality. This can reduce recognition performance.

Preprocessing:
The authors used anchor-based normalization to track relative skeletal motion more accurately.

They also used bilinear interpolation to reconstruct missing keypoints, especially when hand landmarks were not detected.

Model used:
The method used MediaPipe keypoints with preprocessing, reconstruction, normalization, and a recognition model for isolated sign classification.

Evaluation metrics:
The main evaluation metric was classification accuracy.

Results:
The proposed MediaPipe keypoint preprocessing methods improved recognition accuracy by **6.05%**. With data augmentation, the system achieved **83.26% accuracy** on the WLASL dataset. These results show that keypoint normalization, reconstruction, and fixed-length sequence processing can significantly improve isolated sign language recognition.

Contribution:
The main contribution is showing that keypoint preprocessing can significantly improve isolated sign language recognition performance.

3.2.4 Study 4:

Continuous Sign Language Recognition with Correlation Network

Research group and year:
Hu et al., 2023.

Proposed Work:
The authors proposed **CorrNet**, a correlation-based network for continuous sign language recognition. The method focuses on modeling body trajectories across frames to better capture temporal motion patterns in sign videos.

Dataset:
The model was evaluated on several continuous sign language recognition datasets, including **PHOENIX14**, **PHOENIX14-T**, **CSL-Daily**, and **CSL**.

Problem:
The main problem addressed in this study is continuous sign language recognition. Unlike isolated sign recognition, continuous recognition requires recognizing a sequence of signs from a long video without clear boundaries between signs.

Preprocessing:
The method analyzes video sequences and focuses on temporal body trajectories. The goal is to capture local temporal movements that help identify signs over time.

Model used:
The proposed model used a correlation network to model relationships and trajectories across frames. This helps the model focus on meaningful motion instead of treating frames independently.

Evaluation metrics:
Continuous sign language recognition systems are commonly evaluated using Word Error Rate and recognition accuracy depending on the dataset protocol.

Results:
CorrNet was evaluated using **Word Error Rate (WER)**, where a lower value indicates better recognition performance. On **PHOENIX14**, CorrNet achieved **18.8% WER** on the development set and **19.4% WER** on the test set. On **PHOENIX14-T**, it achieved **18.9% WER** and **20.5% WER** on the corresponding evaluation splits. The paper reported state-of-the-art performance on PHOENIX14, PHOENIX14-T, CSL-Daily, and CSL by explicitly modeling body trajectories across frames.

Contribution:
The main contribution is a correlation-based temporal modeling approach that improves continuous sign language recognition by focusing on body trajectories across frames.

3.2.5 Study 5:

Towards Online Continuous Sign Language Recognition and Translation

Research group and year:
Zuo, Wei, and Mak, 2024.

Proposed Work:
The authors proposed an online continuous sign language recognition framework. Their method consists of three main phases: building a sign dictionary, training an isolated sign language recognition model on the dictionary, and using a sliding window approach to recognize signs from continuous input. The paper also extends the online recognition model to translation by integrating a gloss-to-text network.

Dataset:
The paper evaluated the proposed online recognition and translation method on popular continuous sign language recognition and translation benchmarks.

Problem:
The study addressed the problem of high latency in offline continuous sign language recognition. Many offline systems require the whole video before prediction, which is not suitable for online or real-time usage.

Preprocessing:
The system uses a sign dictionary and sliding windows over continuous sign sequences. Each window is processed by an optimized isolated sign recognition model.

Model used:
The proposed framework uses an isolated sign recognition model for online recognition and extends it with a gloss-to-text network for online translation.

Evaluation metrics:
The study uses metrics suitable for continuous sign language recognition and translation, including recognition and translation evaluation metrics depending on the task setting.

Results:
The study used **WER** for online continuous sign language recognition and **BLEU-4** for online sign language translation. For online CSLR, the proposed method achieved **22.2% WER** on the development set and **22.1% WER** on the test set in the reported ablation setting. For online SLT, the method achieved **23.75 BLEU-4** on the PHOENIX-2014T development set and **23.69 BLEU-4** on the test set using a window size of 16. These results show that the online framework can reduce latency while maintaining competitive recognition and translation performance.

Contribution:
The main contribution is the development of an online sign recognition and translation framework that reduces latency and supports gloss-to-text translation.

3.2.6 Study 6:

Sign Language Translation with Sentence Embedding Supervision

Research group and year:
Hamidullah, van Genabith, and España-Bonet, 2024.

Proposed Work:
The authors proposed a sign language translation approach that uses sentence embeddings of the target sentences as a form of supervision during training. Instead of relying mainly on manual gloss annotations, the method uses sentence-level semantic representations to guide translation learning.

Dataset:
The method was evaluated on datasets covering German and American sign languages, including **PHOENIX-2014T** and **How2Sign**.

Problem:
The study addressed the limitation of gloss annotations. Gloss-labeled sign language data is often limited, expensive to create, and inconsistent across datasets.

Preprocessing:
The method uses target sentence embeddings as additional supervision. These embeddings are learned from raw textual data and do not require manual gloss annotation.

Model used:
The paper used a sign language translation framework guided by sentence embedding supervision. The goal was to reduce dependency on gloss labels while improving translation quality.

Evaluation metrics:
The study used translation metrics such as **BLEU**, **chrF**, and **BLEURT**.

Results:
The best reported model, based on **sign2(sem+text)** with an mBART decoder, achieved **24.2 BLEU** on PHOENIX-2014T and **11.7 BLEU** on How2Sign in one setting. The paper also reported a multilingual mBART variant with **24.1 BLEU** on PHOENIX-2014T and **12.0 BLEU** on How2Sign. The authors stated that their best system improved the How2Sign result from about **8 BLEU** to about **12 BLEU**, and improved gloss-free PHOENIX-2014T translation from about **21 BLEU** to about **24 BLEU**.

Contribution:
The main contribution is a sentence embedding supervision strategy that helps sign language translation without depending heavily on manual gloss annotations.

3.2.7 Study 7:

Sign2GPT: Leveraging Large Language Models for Gloss-Free Sign Language Translation

Research group and year:
Wong, Camgoz, and Bowden, 2024.

Proposed Work:
The authors proposed **Sign2GPT**, a gloss-free sign language translation framework that uses large pretrained vision and language models with lightweight adapters. The method also introduces pseudo-gloss pretraining to help the encoder learn better sign representations without requiring manual gloss order annotations.

Dataset:
The model was evaluated on **RWTH-PHOENIX-Weather 2014T** and **CSL-Daily**.

Problem:
The study addressed the lack of large-scale labeled data for sign language translation. It also addressed the challenge of translating long sign videos into spoken-language text without depending on manually annotated glosses.

Preprocessing:
The method used pseudo-gloss pretraining, where automatically extracted pseudo-glosses help the encoder learn sign representations. This reduces the need for manual gloss order annotations.

Model used:
The model used large pretrained vision and language models, lightweight adapters, and pseudo-gloss pretraining. The lightweight adapters were used to make training more efficient under limited sign language dataset sizes and long video sequences.

Evaluation metrics:
The study used translation metrics such as BLEU and ROUGE.

Results:
The Sign2GPT model was evaluated using translation metrics such as **BLEU** and **ROUGE**. With pseudo-gloss pretraining, Sign2GPT achieved **22.52 BLEU-4** and **48.90 ROUGE** on Phoenix14T. On CSL- state-of-the-art by about **4.4 BLEU-4**. These results show the benefit of combining pretrained vision-language models with pseudo-gloss pretraining for gloss-free sign language translation.

Contribution:
The main contribution is a modern LLM-based gloss-free sign language translation framework that integrates visual sign representation and language generation.

Daily, it achieved **15.40 BLEU-4**, improving over the previous gloss-free

3.3 Comparative Analysis of Previous Studies

The following table compares the reviewed studies in terms of publication year, task, method, results, strengths, and limitations.

Table 1 Comparative of Previous Studies in Sign Language Recognition and Translation

Study / System	Year	Task	Method / Model	Results: Metric + Value	Strenghts	Limitation / Gap	Relation to SignStream
ASL Citizen: A Community-Sourced Dataset for Advancing Isolated Sign Language Recognition	2023	Isolated ASL recognition and dictionary retrieval	Community-sourced ASL dataset + supervised baseline	83,399 videos, 2,731 signs, 52 signers; Accuracy = 63%, Recall@10 = 91% on unseen signers	Large and diverse ASL dataset with many signs and signers. Supports signer-independent evaluation.	Focuses on isolated sign recognition and dictionary retrieval only; does not provide full text generation or speech output.	SignStream uses ASL Citizen as the main dataset for training and evaluation.
Natural Language-Assisted Sign Language Recognition	2023	Isolated sign language recognition	Language-aware label smoothing + inter-modality mixup + video-keypoint network	WLASL2000 Top-1 = 61.26%, Top-5 = 91.77%; NMFs-CSL Top-1 = 83.7%, Top-5 = 98.5%	Uses semantic information from gloss labels to improve recognition of visually similar signs.	Improves recognition accuracy but does not provide a complete assistive system with user interaction, text generation, and speech output.	Supports the idea of using language information and NLP after visual recognition in SignStream.
Preprocessing MediaPipe Keypoints with Keypoint Reconstruction and Anchors	2024	Keypoint-based isolated sign recognition	MediaPipe keypoints + reconstruction + anchor-based normalization + augmentation	Accuracy = 83.26% on WLASL; preprocessing improved accuracy by 6.05%	Very close to SignStream because it focuses on MediaPipe keypoints, normalization, reconstruction, and lightweight keypoint-based recognition.	Evaluated mainly on WLASL, which is smaller and different from ASL Citizen; results may not directly transfer to a larger 2,731-class setup.	Reinforces the importance of robust keypoint extraction, normalization, and feature preprocessing in SignStream.
Continuous Sign Language Recognition with Correlation Network	2023	Continuous sign language recognition	CorrNet for body trajectory and temporal correlation modeling	PHOENIX14 WER = 18.8% Dev / 19.4% Test; PHOENIX14-T WER = 18.9% Dev / 20.5% Test	Strong temporal modeling; captures body trajectories across adjacent frames, which is important for motion-based sign understanding.	Designed for continuous sign recognition, which is more complex and harder to deploy than an isolated-sign real-time prototype.	Related to future development of SignStream, especially continuous segmentation and sentence-level recognition.
Towards Online Continuous Sign Language Recognition and Translation	2024	Online continuous recognition and translation	Sign dictionary + isolated recognition model + sliding-window inference + gloss-to-text network	Online CSLR WER = 22.2% Dev / 22.1% Test; Online SLT BLEU-4 = 23.75 Dev / 23.69 Test on P-2014T	Highly relevant to real-time recognition because it targets online inference and integrates gloss-to-text translation.	Requires continuous-sign setup, sliding-window inference, sign dictionary, and more complex online modeling.	Closely related to SignStream live mode, gloss buffering, and NLP-based text generation.
Sign Language Translation with Sentence Embedding Supervision	2024	Sign language translation	Sentence embeddings used as semantic supervision instead of manual gloss annotations	PHOENIX-2014T BLEU = 24.2; How2Sign BLEU = 11.7 for sign2(sem+text) mono	Reduces dependency on manual gloss annotations by using sentence-level semantic supervision.	Focuses on translation datasets and sentence-level training, not isolated ASL gloss recognition.	Supports the NLP direction in SignStream because language-level representations can improve final text output after visual recognition.
Sign2GPT: Leveraging Large Language Models for Gloss-Free Sign Language Translation	2024	Gloss-free sign language translation	Pretrained vision-language models + lightweight adapters + pseudo-gloss pretraining	Phoenix14T BLEU-4 = 22.52, ROUGE = 48.90; CSL-Daily BLEU-4 = 15.40	Modern LLM-based approach that combines computer vision and language modeling for sign translation.	Computationally heavier than SignStream and designed for gloss-free translation, not lightweight desktop recognition.	Supports the idea of integrating visual recognition with NLP, while SignStream uses a more modular pipeline: gloss prediction → NLP refinement → TTS.

3.4 Research Gap and Proposed Innovation

Based on the reviewed studies, several research gaps can be identified. many previous works focus on one part of the problem only. Some studies focus on isolated sign recognition, where the system predicts a single gloss from a video clip. Although this is useful, the output remains a gloss label and does not fully solve the communication problem for users who need readable text or spoken output.

several studies focus on sign language translation, but they often require continuous sign language datasets, sentence-level annotations, gloss annotations, or large pretrained vision-language models. These requirements can make the system more difficult to train, deploy, and use in a lightweight real-time environment.

keypoint-based approaches show strong potential because they reduce the effect of background noise and focus on the signer's hands, body, and face. However, many of these works remain limited to recognition experiments and do not integrate the model into a complete user-facing application.

online sign language recognition remains a challenging problem. Continuous recognition requires sign boundary detection, temporal segmentation, buffering, and stable prediction handling. Many offline systems require the full video before producing output, which increases latency and makes real-time communication harder.

Chapter4

Dataset

4.1 Dataset Description

The dataset used in this project is the **ASL Citizen dataset**, which is a large-scale American Sign Language dataset designed for isolated sign recognition. The dataset contains video samples of different ASL glosses performed by multiple participants. It is suitable for this project because the goal of SignStream is to recognize ASL signs from video input and convert the recognized signs into text and speech.

In the implemented system, the final processed dataset contained **83,387 samples** divided into training, validation, and testing sets. The training set contained **40,154 samples**, the validation set contained **10,304 samples**, and the test set contained **32,929 samples**. The dataset included **2,731 ASL gloss classes**, which makes the task a large-vocabulary classification problem. The dataset also included **52 participants**, where different participants were used across the training, validation, and testing splits. This makes the evaluation more realistic because the model is tested on signers that were not used during training.

Each video was processed as a temporal sequence instead of a single image, because ASL signs are mainly recognized from movement over time. Before training, the videos were preprocessed using **MediaPipe** to extract keypoints from the hands, upper-body pose, and selected facial regions such as the lips, eyes, and eyebrows. These keypoints were then converted into numerical feature sequences. In this project, each sample was represented using a maximum sequence length of **60 frames**, and each frame contained **1725 features**.

Additional preprocessing steps were applied before training. The gloss labels were cleaned using a gloss mapping file to reduce label inconsistencies and spelling variations. The extracted keypoints were normalized to reduce the effect of signer position and camera distance. Motion features such as velocity and acceleration were also included to help the model understand hand movement over time. Finally, the extracted features were saved as **.npz** files, allowing the training process to use precomputed features instead of processing the original videos repeatedly.

Figure 4 Dataset Sampels

Table 2 Dataset Splits

Split	Number of Samples
Training	40,154
Validation	10,304
Test	32,929
Total	83,387

Table 3 Dataset Values

Property	Value
Number of Gloss Classes	2,731
Total Participants	52
Maximum Sequence Length	60 frames
Feature Dimension per Frame	1725
Feature Format	`.npz`

Chapter5

Methodology

5.1 Introduction

This chapter presents the research methodology followed in the development of **SignStream: Real-Time ASL Recognition and Translation System**. It explains the main steps used to design, train, evaluate, and integrate the proposed system.
The chapter begins by discussing traditional approaches for solving sign language recognition problems, including numerical methods and classical machine learning methods. Then, it presents the proposed methodology, including the dataset used, preprocessing steps, feature extraction process, model architecture, training strategy, NLP-based text processing,Text-to-Speech module, and system pipeline.
The purpose of this chapter is to describe how the proposed system was built and why each main component was selected. It also explains how the visual recognition model, language processing stage, speech output, and user interface are organized within the complete SignStream workflow.

5.2 Traditional Methods for Solving the Problem

Before modern deep learning techniques became dominant, sign language recognition was commonly addressed using numerical rules, handcrafted features, and classical machine learning algorithms. These approaches were useful for simple gestures and small datasets, but they had clear limitations when applied to large-vocabulary sign language recognition.
Traditional methods usually follow this general process:
Video Input → Manual Feature Extraction → Rule-Based or Classical Classifier → Predicted Sign
The main weakness of these methods is that they depend heavily on manually designed features. In sign language recognition, this is difficult because signs vary between signers in speed, body position, hand movement, and facial expression. Therefore, a method that works well for one signer or one controlled environment may fail when used with different signers or real-world videos.
In this project, traditional methods are discussed as a baseline understanding, but the final proposed solution uses deep learning because the problem requires learning complex temporal patterns from large-scale data.

5.2.1 Numerical Methods

Numerical methods in sign language recognition depend mainly on mathematical operations applied directly to video frames, motion signals, or extracted body landmarks. These methods include techniques such as frame differencing, motion thresholding, optical flow, hand trajectory analysis, distance measurement between body landmarks, joint angle calculation, velocity estimation, and acceleration estimation. For example, frame differencing and motion thresholding can be used to detect when a sign starts and ends, while hand trajectory analysis can describe the movement path of the signer's hands over time. Similarly, distances between the wrist, shoulders, face, and body center can help describe the spatial location of a sign.
These methods are useful because sign language depends strongly on motion, hand position, body posture, and facial expression. For example, the change in hand position between two

consecutive frames can be used to estimate movement velocity, while the change in velocity can be used to estimate acceleration. Such numerical features can support sign segmentation and provide a basic description of the signer's movement.
However, numerical methods alone are not sufficient for robust ASL recognition. They are highly sensitive to lighting conditions, camera angle, signer distance, hand occlusion, motion blur, and differences between signers. They also depend on manually designed rules and thresholds, which makes them difficult to generalize across different users and real-world scenarios. In addition, many ASL signs are visually similar and cannot be distinguished using simple motion or distance measurements only. Therefore, numerical methods can be useful as preprocessing or supporting features, but they are not enough as the main recognition approach for a large-vocabulary ASL recognition system.

5.2.2 Machine Learning Methods

Classical machine learning methods improved traditional sign language recognition approaches by learning patterns from extracted features instead of relying only on fixed numerical rules. In these methods, handcrafted features are first extracted from images, video frames, or landmark sequences. These features may include hand position, hand trajectory, movement direction, velocity, acceleration, joint angles, distances between body landmarks, optical flow descriptors, or shape-based features. After feature extraction, classifiers such as Support Vector Machine, k-Nearest Neighbors, Random Forest, Logistic Regression, or Hidden Markov Models can be used to classify the input into predefined sign classes.
These methods can be useful for simple and controlled sign recognition scenarios. For example, they may recognize a small number of isolated hand gestures when the camera position, lighting conditions, background, and signer distance are fixed. Hidden Markov Models can also be used to model temporal movement patterns because sign language depends on motion over time. However, the performance of these methods depends heavily on the quality of the manually extracted features. If the signer moves at a different speed, the hand is partially occluded, the lighting changes, or the same sign is performed with a different style, the handcrafted features may not represent the sign accurately.
For this reason, the proposed project uses a deep learning-based pipeline instead of relying only on numerical or classical machine learning methods. Deep learning models can automatically learn more powerful temporal representations from data, which makes them more suitable for large-vocabulary ASL recognition. In this project, the system extracts keypoint sequences from video and uses a temporal deep learning model based on CNN and Transformer layers to recognize ASL glosses. This approach is more appropriate for the SignStream system because it can handle complex motion patterns, signer variation, and a large number of sign classes.

5.3 Proposed Model Architecture and Training Methodology

The proposed model is designed to classify ASL glosses from temporal keypoint sequences. Each video is first converted into a sequence of extracted features. Then, this sequence is passed to a deep learning model that learns motion patterns and predicts the corresponding ASL gloss.
The input to the model is represented as:

$$X \in R^{T \times F}$$

where:

- TTT is the number of frames in the sequence.
- FFF is the feature dimension per frame.

In this project, each sample is represented using a maximum of **60 frames**, and each frame contains **1725 features**. Therefore, the input shape used by the model is:

60×1725

The proposed visual recognition model is called **SignTransformer V3-Large**. It combines temporal convolutional layers and Transformer encoder layers. The temporal CNN layers capture short-term motion patterns between neighboring frames, while the Transformer layers capture long-range temporal relationships across the full sign sequence.
The general architecture can be summarized as follows:

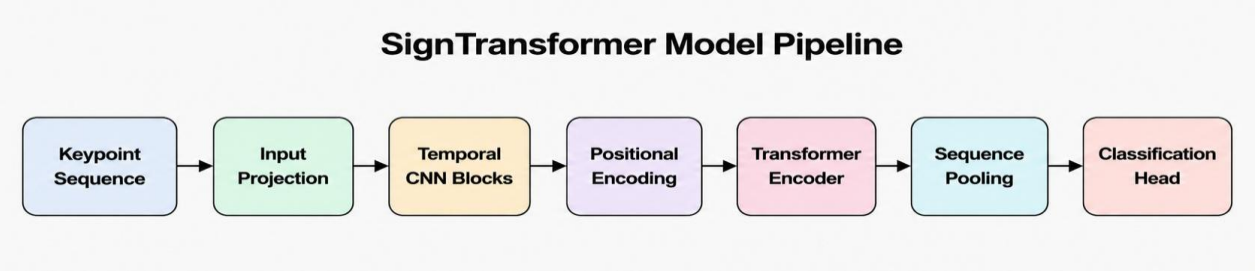

Figure 5 SignTransformer model Pipeline

5.3.1 Input Representation

The input representation is based on keypoint features extracted from video frames. Instead of using raw pixels, the system uses structured human landmarks from the hands, upper-body pose, and selected facial regions.
Each frame is converted into a feature vector that includes:

- Hand landmarks.
- Upper-body pose landmarks.
- Selected facial landmarks.
- Normalized coordinates.
- Velocity features.
- Acceleration features.
- Missing landmark indicators.

This representation reduces the influence of background details and allows the model to focus on the signer's motion and body structure.

5.3.2 SignTransformer V3-Large Architecture

The SignTransformer V3-Large model consists of several main components: input projection, temporal CNN residual blocks, positional encoding, Transformer encoder layers, pooling, and classification head.

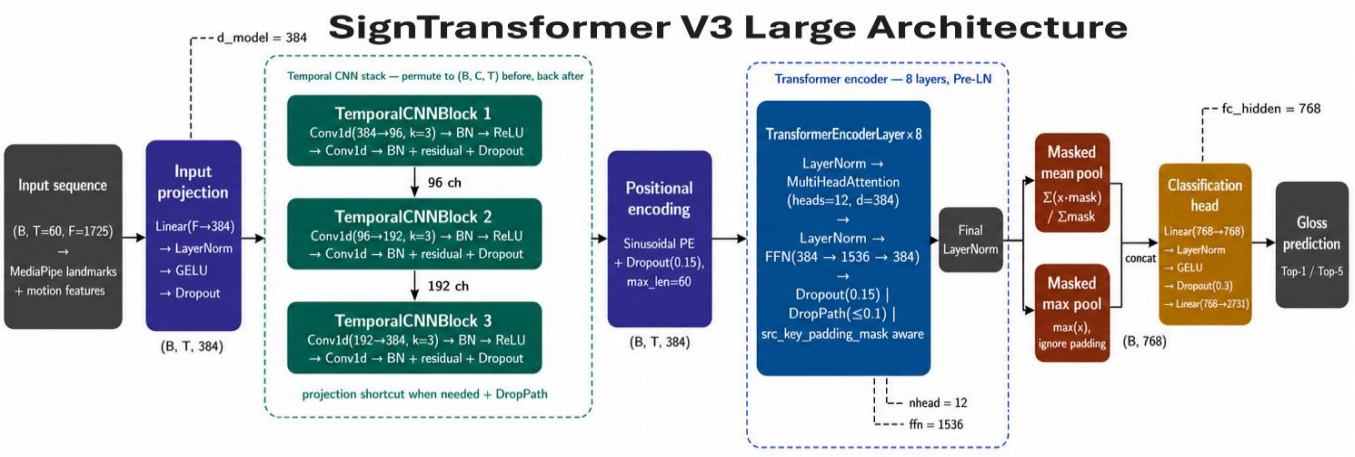

Figure 6 SignTransformer Architecture

5.3.2.1 Input Projection Layer

The input feature vector has a high dimension because it contains landmarks and motion-related features. Therefore, the first stage of the model projects the original input into a compact embedding space.
The input projection maps the original feature dimension into:

$$\mathbf{d_{model=}384}$$

This layer includes:

- Linear layer.
- Layer Normalization.
- GELU activation.
- Dropout.

This stage helps convert raw numerical features into a learned representation suitable for temporal modeling.

5.3.2.2 Temporal CNN Residual Blocks

After the input projection, the sequence is passed through temporal CNN residual blocks. These blocks apply one-dimensional convolution along the time dimension. Their purpose is to capture short-term movement patterns such as hand transitions, local motion direction, and short pauses.

The temporal CNN stack uses the following channel sizes:

[96,192,384]

Residual connections are used to improve training stability. A residual block can be represented as:

$$y=F(x)+x$$

where xxx is the input and $F(x)$ is the transformation learned by the convolutional block. Residual connections help the model preserve useful information and make deeper networks easier to train.

5.3.2.3 Positional Encoding

Transformer models do not naturally understand the order of frames. Therefore, positional encoding is added to the sequence before the Transformer encoder.
Positional encoding gives each frame information about its position in the sequence. This is important because the meaning of a sign depends on the order of movement. The same hand position may have a different meaning if it appears at the beginning or the end of the sign.
This stage helps the model distinguish between early, middle, and late parts of the sign sequence.

5.3.2.4 Transformer Encoder

The Transformer encoder is the main temporal modeling component of the proposed model. It uses self-attention to allow each frame in the sequence to compare itself with other frames and determine which frames are most important for classification.

Table 4 SignTransformer V3-Large Transformer Encoder Configuration

Component	Value
Transformer layers	8
Attention heads	12
Model dimension	384
Feed-forward dimension	1536
Transformer type	Pre-Layer Normalization
Activation function	GELU
Regularization	Dropout and DropPath

The self-attention mechanism is computed as:

$$Attention(Q, K, V)=softmax(\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where:

- $\boldsymbol{Q}$ is the query matrix.
- $\boldsymbol{K}$ is the key matrix.
- $\boldsymbol{V}$ is the value matrix.
- $d_k$is the key dimension.

Self-attention is useful for ASL recognition because the most important frames are not always located at the same position in every video. Some signs depend on the first motion, others depend on the final hand shape, and others depend on the transition between hand positions.

5.3.2.5 Sequence Pooling and Classification Head

After the Transformer encoder, the model produces a feature representation for each frame. These frame-level outputs must be converted into one vector before classification.
The model uses mask-aware mean pooling and max pooling.
Mean pooling captures the general information across the whole sign:

$$h_{mean} = \frac{1}{T}\sum_{t=1}^{T} h_t$$

Max pooling captures the strongest and most distinctive temporal activation:

$$h_{max} = \max_{t}(h_t)$$

The final sequence representation is obtained by concatenating both:

$$\boldsymbol{h} = [h_{mean}; h_{max}$$

Because some samples are padded to reach 60 frames, mask-aware pooling is used to ignore padded positions.
The classification head maps the final representation to the number of ASL gloss classes. In this project, the model predicts one class from: **2731**
ASL gloss classes.
The classification head consists of:

- Linear layer.
- Layer Normalization.
- GELU activation.
- Dropout.
- Final Linear layer.

The predicted class is selected using:

$$h_{max} = \underset{t}{argmax} z_i$$

where z_i is the output logit of class $\boldsymbol{i}$.

5.4 Loss Function

The recognition task is a multi-class classification problem because each video belongs to one ASL gloss class. Therefore, the main loss function used is Cross-Entropy Loss.
First, the model outputs logits for all classes. These logits are converted into probabilities using the Softmax function:

$$\boldsymbol{P}\,(c_i) = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}}$$

where:

- z_i is the logit of class $\boldsymbol{i}$.
- $\boldsymbol{k}$ is the number of classes.

The Cross-Entropy Loss is defined as:

$$\boldsymbol{L} = -\sum_{i=1}^{k} y_i log\ (\hat{y}_i)$$

where:

- y_i is the true label.
- $\hat{y}_i$ is the predicted probability.
- $\boldsymbol{k}$ is the number of classes.

If the model gives a high probability to the correct class, the loss becomes small. If it gives a low probability to the correct class, the loss becomes large.

In this project, label smoothing is also used to reduce overconfidence. Instead of forcing the model to assign all probability to the correct class, label smoothing distributes a small portion of probability to other classes. This is useful because some ASL signs are visually similar.
The label smoothing value used in the proposed model is:

$$\boldsymbol{\epsilon = 0.15}$$

5.4.1 Training Algorithms and Optimization Strategy

The model is trained using an AdamW-style optimizer. AdamW is suitable for deep neural networks because it uses adaptive learning rates and applies weight decay regularization to reduce overfitting.

Table 5 Training Configuration and Hyperparameters

Parameter	Value
Model	SignTransformer V3-Large
Input dimension	1725
Output classes	2731
Maximum sequence length	60
Model dimension	384
Transformer layers	8
Attention heads	12
Feed-forward dimension	1536
Batch size	32
Gradient accumulation	4
Effective batch size	128
Learning rate	3e-4
Weight decay	0.05
Warmup epochs	4
Scheduler	Cosine schedule
Label smoothing	0.15
Mixup alpha	0.3
EMA decay	0.999
Gradient clipping	1.0
Mixed precision	Enabled

Several training techniques are used to improve stability and generalization.

Gradient Accumulation:
Gradient accumulation is used to simulate a larger batch size without exceeding GPU memory. The model processes several smaller batches and updates the weights after accumulating gradients.

Cosine Learning Rate Schedule:
The learning rate starts with a warmup phase and then gradually decreases using a cosine schedule. This helps stabilize training during the early epochs.

Gradient Clipping:
Gradient clipping limits the maximum gradient norm to prevent unstable updates and exploding gradients.

Mixed Precision Training:
Mixed precision training reduces GPU memory usage and speeds up training by using lower-precision operations when possible.

Exponential Moving Average:
EMA maintains a smoothed copy of the model weights. This usually improves evaluation stability and final model performance.

Mixup:
Mixup combines training samples and their labels to produce smoother decision boundaries and reduce overfitting.
Dropout and DropPath:
Dropout and DropPath are used as regularization techniques to reduce overfitting and improve generalization.

5.4.2 Data Augmentation

Data augmentation is used during training to make the model more robust to signer variation and noisy keypoint extraction. Since different signers may perform the same sign with different speed, position, and movement style, augmentation helps the model generalize better.
The augmentation strategy includes:

Table 6 Data Augmentation Strategy

Augmentation	Purpose
Spatial noise	Simulates small landmark detection errors
Horizontal flip	Improves robustness to signer orientation
Time warping	Simulates temporal movement variation
Time masking	Makes the model robust to missing frames
Speed perturbation	Simulates fast and slow signing
Random erasing	Reduces dependency on specific features

These augmentations are important because real-world sign language recognition must handle different signers, speeds, and motion styles

5.5 NLP-Based Gloss-to-Text Processing

The visual recognition model produces ASL gloss predictions. A gloss is a simplified written label that represents the meaning of a sign. However, glosses are not always equivalent to natural English sentences. In many cases, the predicted gloss sequence may be incomplete, grammatically weak, or different from normal English word order. Therefore, an NLP-based processing stage is included in the SignStream system to improve the final textual output.
The purpose of the NLP component is to convert the recognized gloss sequence into a clearer and more readable sentence. This stage is important because the final user does not only need isolated gloss labels; the user needs understandable text that can later be converted into speech. For example, a sequence of predicted glosses may contain words in a simplified or non-grammatical order. The NLP module helps refine this sequence and produce a more natural sentence.
The NLP stage receives the accepted glosses from the visual recognition module. These glosses

are collected through the buffering mechanism, which prevents repeated predictions of the same sign in live mode. After a stable sequence of glosses is formed, the NLP module processes it and generates the final text output.
The general NLP workflow can be summarized as follows:

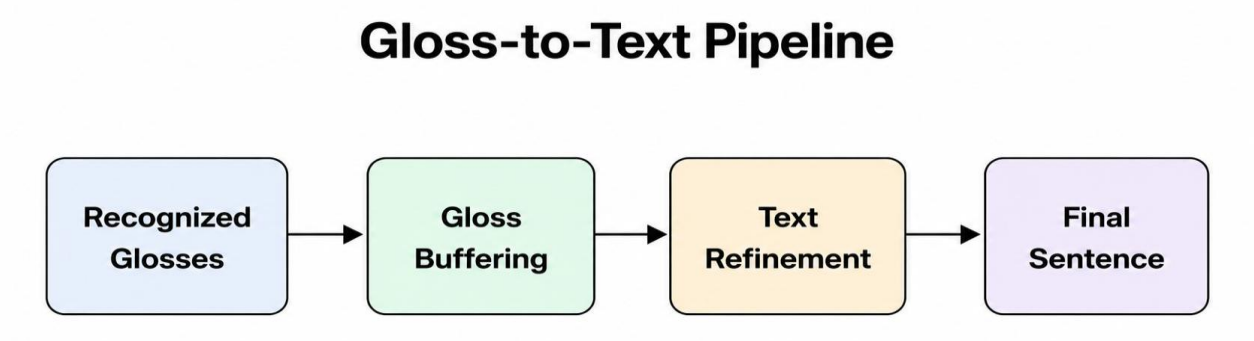

Figure 7 Gloss-to-Text Piprline

The NLP module contributes to the system in several ways:

1. It converts isolated gloss predictions into more readable text.
2. It reduces the effect of repeated or unstable predictions.
3. It improves the grammatical structure of the output sentence.
4. It prepares the text for the text-to-speech module.
5. It makes the system more useful as an assistive communication tool.

In this project, the NLP component is treated as a post-processing stage after visual recognition. The visual model is responsible for recognizing signs from video, while the NLP module is responsible for improving the language form of the recognized output. This separation makes the system easier to design and evaluate because each component has a clear role.
The NLP stage is especially important in real-time usage. In live mode, the system continuously receives predictions from the recognition model. Without a language processing step, the output may appear as repeated or disconnected glosses. By using gloss buffering and text refinement, the system can produce a cleaner sentence that is easier for the user to understand.
The output of the NLP module is then sent to the text-to-speech module. This means that the final text can be spoken aloud, completing the communication pipeline from sign language video to speech output.

5.6 Text-to-Speech Module

The Text-to-Speech module is the final output stage in the SignStream system. After the visual recognition model predicts ASL glosses and the NLP module converts them into readable text, the TTS module converts the final text into spoken audio. This step is important because the goal of the project is not only to recognize signs, but also to support communication between ASL users and people who do not understand sign language.
The TTS module receives the final sentence generated by the NLP stage as input. Then, it uses a

speech synthesis engine to produce an audible voice output. The general workflow of the module can be represented as follows:

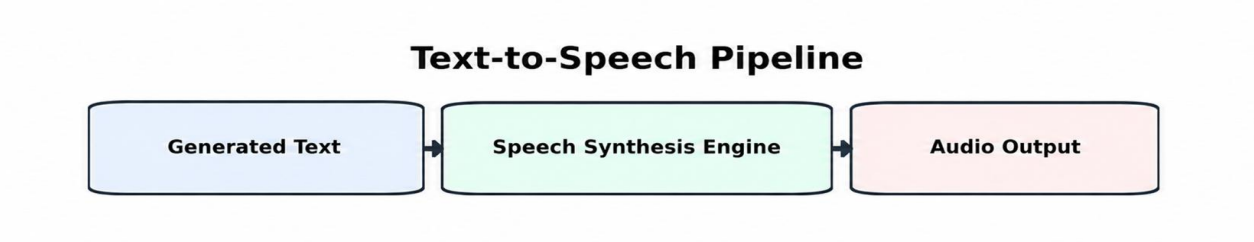

Figure 8 Text-to-Speech Pipeline

In the implemented system, an offline TTS engine is used to generate speech from text. This makes the system more practical because it can produce voice output without requiring an internet connection. The module is integrated into the user interface, where the user can press the speak button after generating the final text. The system then reads the recognized sentence aloud. The role of the TTS module can be summarized in the following points:

1. It converts the final recognized text into spoken language.
2. It makes the system more useful for real communication scenarios.
3. It allows non-signers to hear the meaning of the performed signs.
4. It completes the full translation pipeline from video input to audio output.
5. It improves accessibility by providing both text and voice output.

The TTS module is not trained as a separate deep learning model in this project. Instead, it is used as an integrated speech synthesis component. The main focus of the project is the visual recognition model and the complete system pipeline. However, adding TTS gives the system a more practical assistive function because the output is not limited to visual text on the screen.
In live usage, the TTS stage is applied after the system accepts stable predictions and generates the final text. This prevents the system from speaking repeated or unstable predictions. Therefore, the TTS module depends on the previous stages: visual prediction, gloss buffering, and NLP-based text refinement.
The complete final stage can be described as:

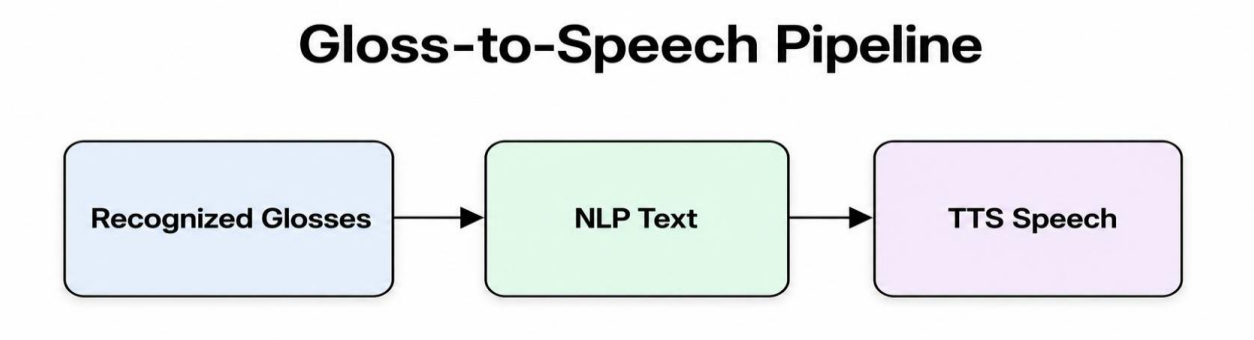

Figure 9 Gloss-to-Speech Pipeline

This integration makes SignStream closer to a real assistive communication prototype because it converts sign language video into both readable and audible output.

Table 7 Roles of the Main SignStream System Components.

Component	Input	Processing	Output
Keypoint extraction	Video frames	MediaPipe landmark extraction	Hand, pose, and selected face keypoints
Visual recognition model	Keypoint sequence	Temporal CNN + Transformer Encoder	Predicted ASL gloss
Gloss buffering	Continuous predictions	Filtering and confidence-based acceptance	Stable gloss sequence
NLP module	Accepted gloss sequence	Gloss-to-text refinement	Readable sentence
TTS module	Generated sentence	Offline speech synthesis	Spoken audio
User interface	System outputs	Display and interaction	Predictions, text, speech, and history

5.7 SignStream Pipeline

The SignStream ASL Recognition pipeline was designed to analyze video input and recognize American Sign Language signs. The workflow consists of the following main steps.

1. Video Input:
The input to the system is either a live webcam stream or an uploaded video. Each video contains an ASL sign performed by a signer. The system supports both modes to make the project suitable for real-time demonstration and offline testing.

2. Frame Extraction:
The input video is read as a sequence of frames. Each frame represents a specific moment in the sign movement. This step is important because ASL signs are not recognized from a single image only, but from the movement pattern across time.

3. Frame Preprocessing:
Before feature extraction, the frames are prepared by resizing and converting them into a suitable format for processing. This helps make the input consistent even when videos have different resolutions or frame sizes.

4. Keypoint Extraction:
Each frame is processed using MediaPipe to extract human landmarks. The system focuses on the most important regions for sign language recognition, including both hands, upper-body pose, and selected facial regions such as the lips, eyes, and eyebrows. This step allows the model to focus on the signer's body motion instead of irrelevant background information.

5. Feature Construction:
The extracted landmarks are converted into numerical feature vectors. These features include keypoint coordinates, normalized positions, velocity, acceleration, and missing landmark indicators. In the implemented system, each frame is represented by **1725 features**.

6. Sequence Generation:
The frame-level features are combined into a temporal sequence. Each video sample is represented using a maximum of **60 frames**, which provides a consistent input structure for the model. If a video has fewer frames, padding is applied. If it has more frames, sampling or trimming is used.

7. Feature Normalization:
The extracted keypoints are normalized to reduce the effect of camera distance, signer position, and body size differences. This helps the model focus on the relative movement and shape of the sign rather than the absolute position of the signer in the frame.

8. Temporal Model Input:
The processed keypoint sequence is passed to the SignTransformer V3-Large model. The input shape used by the model is:

60×1725

where 60 represents the maximum number of frames and 1725 represents the feature dimension of each frame.

9. Temporal Feature Learning:
The model first uses temporal CNN layers to capture short-term motion patterns between neighboring frames. Then, Transformer encoder layers are used to learn long-range temporal relationships across the full sign sequence. This helps the model understand both local hand movements and the overall structure of the sign.

10. Gloss Classification:
After temporal modeling, the classification head predicts the ASL gloss class. The model produces probabilities over **2731 gloss classes**. The system can display the Top-1 prediction and the Top-5 most likely predictions with confidence scores.

11. Gloss Buffering:
In live mode, predictions are generated continuously. Therefore, a buffering mechanism is used to avoid repeated predictions of the same sign. The buffer keeps only stable and accepted predictions, which helps produce a cleaner sequence of recognized glosses.

12. NLP-Based Text Generation:
After gloss classification and buffering, the accepted glosses are passed to the NLP module. This module converts the recognized gloss sequence into a clearer textual sentence. This step is necessary because ASL glosses do not always follow the same structure as written English. The NLP stage improves readability, reduces repeated predictions, and prepares the sentence for speech synthesis.

13. Text-to-Speech:
After the NLP module generates the final sentence, the text is passed to the Text-to-Speech module. This module converts the written sentence into spoken audio using an offline speech synthesis engine. This step allows the recognized ASL signs to be heard as speech, making the system more practical for communication with people who do not understand sign language.

14. Final Output:
The final output of the system includes the predicted ASL gloss, confidence score, Top-5 predictions, generated text, and optional speech output. The user interface also displays useful information such as FPS, latency, accepted words, and session history.

5.8 Dataset Used and Preprocessing

5.8.2 Gloss Cleaning

The original dataset contains gloss variations and inconsistent labels. Therefore, a gloss mapping file is used to normalize labels across training, validation, and test splits.
This step reduces label noise and ensures that equivalent glosses are treated consistently during training and evaluation.

5.8.3 Keypoint Extraction

Before training, the videos were preprocessed using MediaPipe to extract keypoints from the hands, upper-body pose, and selected facial regions such as the lips, eyes, and eyebrows. These keypoints were then converted into numerical feature sequences.
The system focuses on selected facial regions instead of using the full face mesh because the full face representation increases the feature dimension and computational cost. The selected regions are more relevant for sign language recognition and facial expression cues.
The extracted keypoints include:

- Hand landmarks.
- Upper-body pose landmarks.
- Selected face landmarks.
- Motion features such as velocity.
- Motion features such as acceleration.

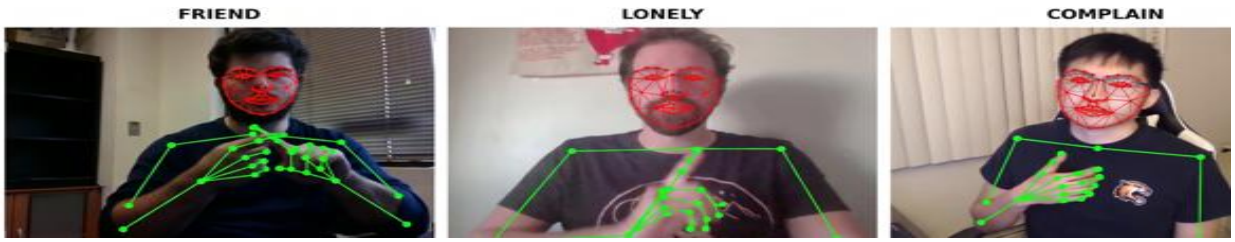

Figure 10 Keypoints

5.8.4 Feature Construction and Sequence Standardization

After keypoint extraction, the landmarks are converted into fixed-format numerical features. In this project, each frame contains: **F=1725** features.
Each sample is represented using a maximum sequence length of: **T=60**
If the sequence is shorter than 60 frames, padding is applied. If the sequence is longer than 60 frames, sampling or trimming is applied. A mask is used to distinguish real frames from padded frames so the model does not learn from padding.
The final input format is: $X \in R^{60\times1725}$
This fixed shape allows the model to process samples in batches during training.

Table 8 Feature Vector Composition Used in SignStream

Feature Component	Description	Purpose
Hand landmarks	Keypoints extracted from both hands	Captures hand shape and hand motion
Upper-body pose landmarks	Pose landmarks related to arms, shoulders, and upper body	Captures body posture and arm movement
Selected face landmarks	Facial regions such as lips, eyes, and eyebrows	Supports facial expression cues relevant to ASL
Normalized coordinates	Landmark coordinates normalized across frames	Reduces the effect of camera position and signer distance
Velocity features	First-order temporal differences between frames	Captures movement direction and speed
Acceleration features	Second-order temporal changes	Captures sudden motion changes
Final frame vector	1,725 features per frame	Represents each video frame numerically
Final sequence shape	60 × 1725	Represents one fixed-length model input sample

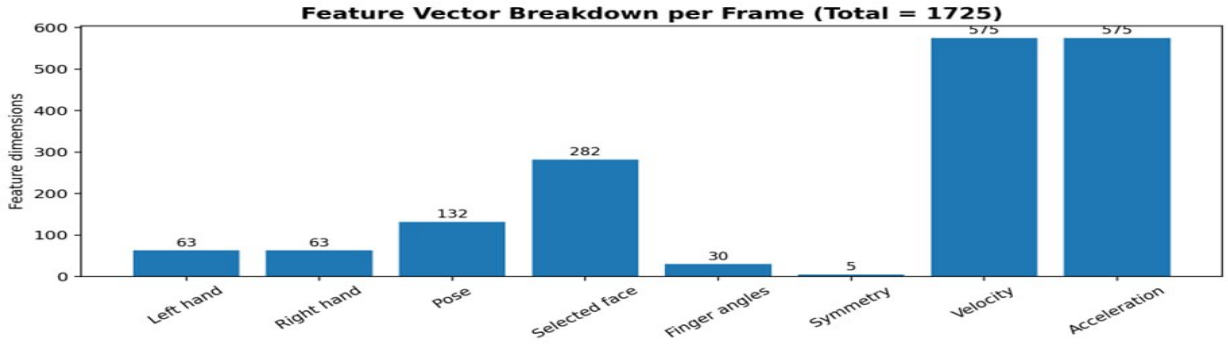

Figure 11 Feature Vector

5.8.5 Feature Saving

The extracted features were saved as `.npz` files. This allows the training process to use precomputed features instead of processing the original videos repeatedly.
This design is important because feature extraction from thousands of videos is computationally expensive. Saving the extracted features also makes the training process faster and more reliable, especially in cloud notebook environments such as Kaggle, where sessions may stop after a limited time.

5.8.6 Tools and Technologies Used

The project uses the following tools and technologies:

Table 9 Software Tools and Technologies Used in SignStream

Tool / Technology	Purpose
Python	Main programming language
PyTorch	Deep learning model training
MediaPipe	Keypoint extraction
OpenCV	Video reading and frame processing
NumPy	Numerical processing
Pandas	Metadata and CSV handling
Kaggle	Training and feature extraction environment
PyQt5	Desktop user interface
pyttsx3	Offline text-to-speech
Transformers / T5	Gloss-to-text processing
Matplotlib	Visualization and plotting

These tools were selected because they support the main requirements of the project: video processing, keypoint extraction, deep learning training, evaluation, interface design, and speech output.

5.9 Chapter Summary and Research Contribution

This chapter presented the methodology used to develop **SignStream: Real-Time ASL Recognition and Translation System**. It explained why traditional numerical and classical machine learning methods are limited for large-vocabulary ASL recognition, especially because they depend on handcrafted features and manually designed rules that do not generalize well across different signers and motion patterns.
The chapter then described the proposed visual recognition pipeline. Video input is converted into structured keypoint sequences using MediaPipe landmarks from the hands, upper-body pose, and selected facial regions. Each sample is represented using a fixed input shape of **60 × 1725** and passed to the **SignTransformer V3-Large** model, which combines temporal CNN blocks

and Transformer encoder layers to learn both local motion patterns and long-range temporal dependencies.

The chapter also explained the training methodology, including the loss function, label smoothing, optimization strategy, regularization techniques, and data augmentation methods used to improve model generalization. In addition, it described the NLP-based gloss-to-text stage, which converts accepted gloss predictions into clearer English text, and the Text-to-Speech module, which converts the final text into spoken audio.

The main contribution of this methodology is the integration of computer vision, keypoint extraction, temporal deep learning, gloss buffering, NLP-based text refinement, TTS synthesis, and a desktop user interface into one complete assistive prototype. The next chapter presents the experimental results, evaluation metrics, model performance, and comparison with the baseline model.

Chapter6

Results and Discussion

6.1 Introduction

This chapter presents the experimental results and discussion of the proposed **SignStream: Real-Time ASL Recognition and Translation System**. It provides an overview of the evaluation process, the tools used during training and testing, and the metrics adopted to measure system performance.

The chapter focuses mainly on evaluating the visual recognition model, since it is the core component responsible for predicting ASL glosses from extracted keypoint sequences. It also discusses the performance of the proposed model, compares it with the previous baseline model, and analyzes the results from different perspectives, including validation performance, test performance, Top-1 and Top-5 accuracy, and per-signer behavior.

In addition, this chapter briefly presents the results of the NLP stage and the behavior of the user interface during real-time and uploaded-video demonstrations. The goal of this chapter is to determine how effectively the proposed system performs and to identify the strengths and limitations of the implemented approach.

6.2 Tools Used and Evaluation Methods

This chapter presents the experimental results of the proposed **SignStream** system. The evaluation focuses mainly on the visual recognition model because it is the core component responsible for predicting ASL glosses from video-based keypoint sequences. The chapter also discusses the tools used during training and evaluation, the metrics used to measure model performance, the results of the proposed model, and a comparison with the previous baseline model.
The model was trained and evaluated using pre-extracted keypoint features from the ASL Citizen dataset. Each sample was represented as a temporal sequence with a maximum length of 60 frames and 1725 features per frame. The classification task included 2,731 ASL gloss classes, making it a large-vocabulary recognition problem.
The evaluation was performed using classification metrics suitable for multi-class sign recognition. Since the task contains a large number of classes, relying only on Top-1 accuracy may not fully describe the model behavior. Therefore, both Top-1 accuracy and Top-5 accuracy were used.

6.2.1 Top-1 Accuracy

Top-1 accuracy measures whether the highest-confidence predicted class is equal to the true class. It is the strictest metric used in this project.

$$\textbf{\textit{Top-1}}=\frac{Numberofcoorecttop-1predictions}{totalnumberofsampels}\times 100$$

If the model predicts the correct ASL gloss as its first choice, the prediction is counted as correct. This metric is important because it reflects the direct output that would normally be shown to the user as the main prediction.

6.2.2 Top-5 Accuracy

Top-5 accuracy measures whether the true class appears among the five highest-confidence predictions.

$$\textbf{\textit{Top-5}}=\frac{Numberofsampelswheretrueclassis\in Top-5}{totalnumberofsampels}\times 100$$

Top-5 accuracy is especially useful in large-vocabulary classification problems. In this project, the model predicts one class among 2,731 ASL glosses. Some signs are visually similar, so the correct gloss may not always be the first prediction but may still appear among the top candidates. Therefore, Top-5 accuracy gives a better view of whether the model has learned useful visual representations.

6.2.3 Validation and Test Evaluation

The validation set was used during training to monitor model performance and select the best checkpoint. The test set was used for final evaluation after training. This separation is important because the test set provides a more realistic estimate of the model's generalization ability.
The evaluation also included EMA-based testing. EMA, or Exponential Moving Average, maintains a smoothed version of model weights during training. The EMA version of the model was used for final test evaluation because it usually provides more stable results.

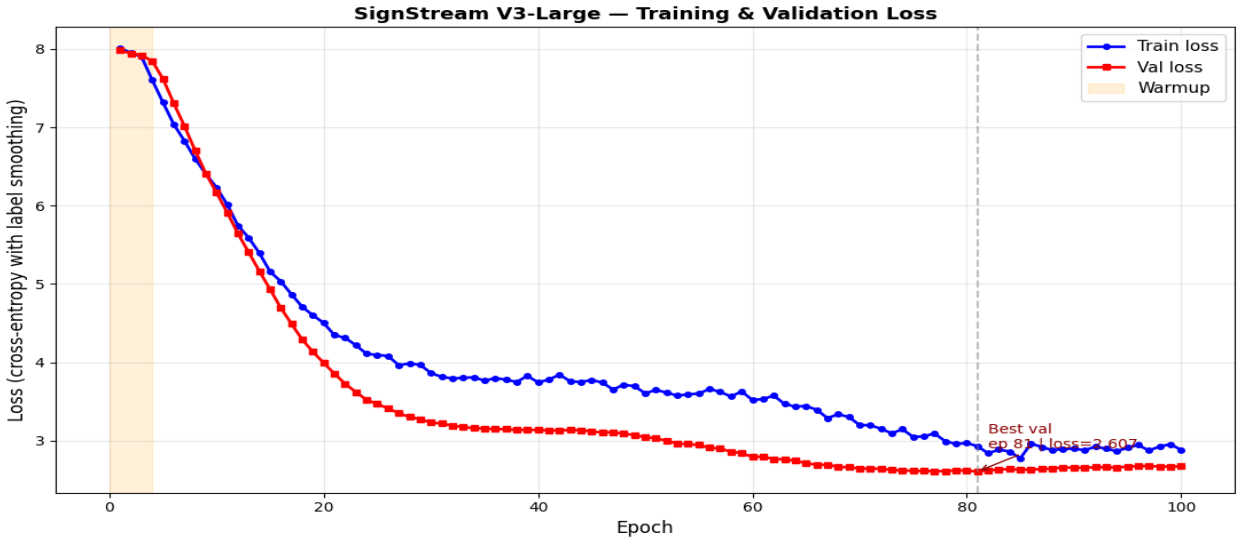

Figure 12 Train & Validation Loss

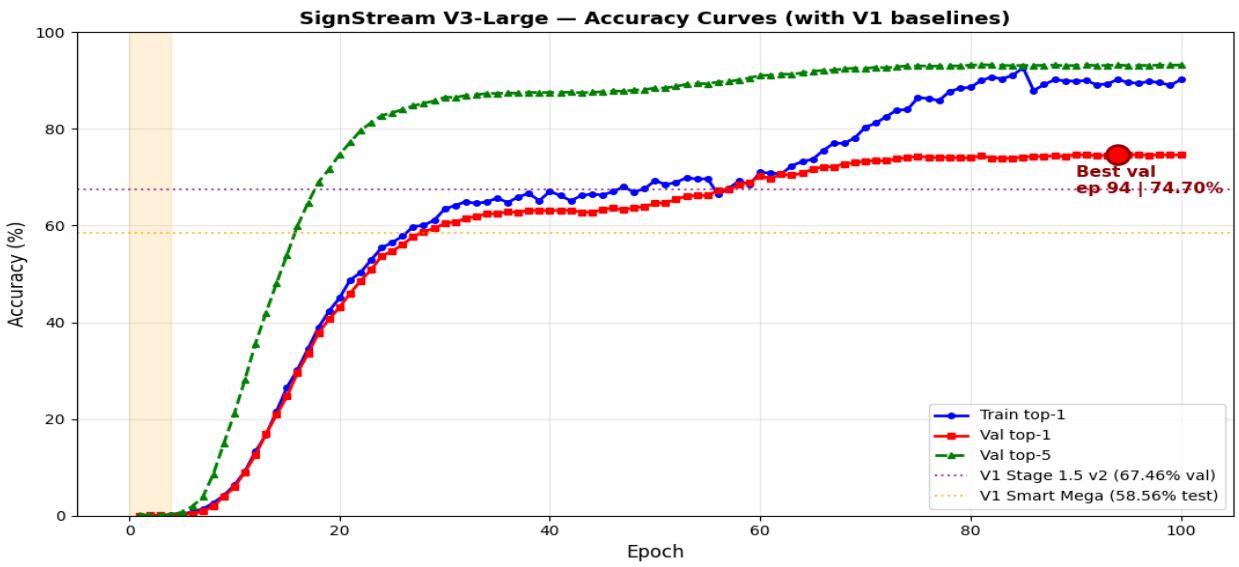

Figure 13 Accuracy

6.3 Results of the Proposed Model

The proposed model used in this project is **SignTransformer V3-Large**. It was trained on the processed ASL Citizen feature dataset using 2,731 gloss classes. The final dataset contained 40,154 training samples, 10,304 validation samples, and 32,929 test samples.
The model achieved its best validation performance at epoch 94. The final results are shown in Table 5.1.

Table 5.1. Final Validation and Test Results of SignTransformer V3-Large

Metric	Result
Best validation Top-1 accuracy	74.70%
Best validation Top-5 accuracy	93.23%
Test Top-1 accuracy with EMA	67.40%
Test Top-5 accuracy with EMA	89.31%
Number of classes	2,731
Test samples	32,929

The validation results show that the model learned strong discriminative patterns from the training data. The validation Top-1 accuracy reached **74.70%**, while the validation Top-5 accuracy reached **93.23%**. This indicates that in most validation cases, the correct gloss was included among the top five predictions.

On the test set, the model achieved **67.40% Top-1 accuracy** and **89.31% Top-5 accuracy**. Since the test set contains unseen participants, this result shows that the model can generalize to signers

that were not used during training. The difference between validation and test accuracy is expected because signer-independent testing is more difficult than validation on familiar or less challenging distributions.
The high Top-5 accuracy is important for the SignStream system because the user interface can display the top candidate signs. This allows the system to provide additional useful information even when the first prediction is uncertain.

6.3.1 Interpretation of the Results

The results show that the proposed model is effective for large-vocabulary ASL recognition. Achieving **67.40% Top-1 accuracy** on 2,731 classes is a strong result because the model must distinguish between many visually similar glosses. The **89.31% Top-5 accuracy** also indicates that the model frequently ranks the correct class among its most confident predictions.
The performance difference between Top-1 and Top-5 suggests that many errors occur between similar signs. This is expected in ASL recognition because different signs may share similar hand shapes, movement directions, or body locations. In these cases, the model may still understand the general visual pattern but select a similar gloss as the first prediction.
The results also show that keypoint-based temporal modeling is suitable for this task. Instead of processing raw video frames, the model uses structured keypoint sequences, which reduces background noise and focuses learning on hand, body, and selected facial motion.

6.3.2 Per-Signer Performance Analysis

A per-signer analysis was performed on the test set to study how well the model generalizes across different participants. This is important because signers may perform the same sign with different speed, body posture, hand position, and motion style.
The proposed model improved performance for all test signers compared with the previous baseline. The average improvement was **+8.84 percentage points** in test Top-1 accuracy.

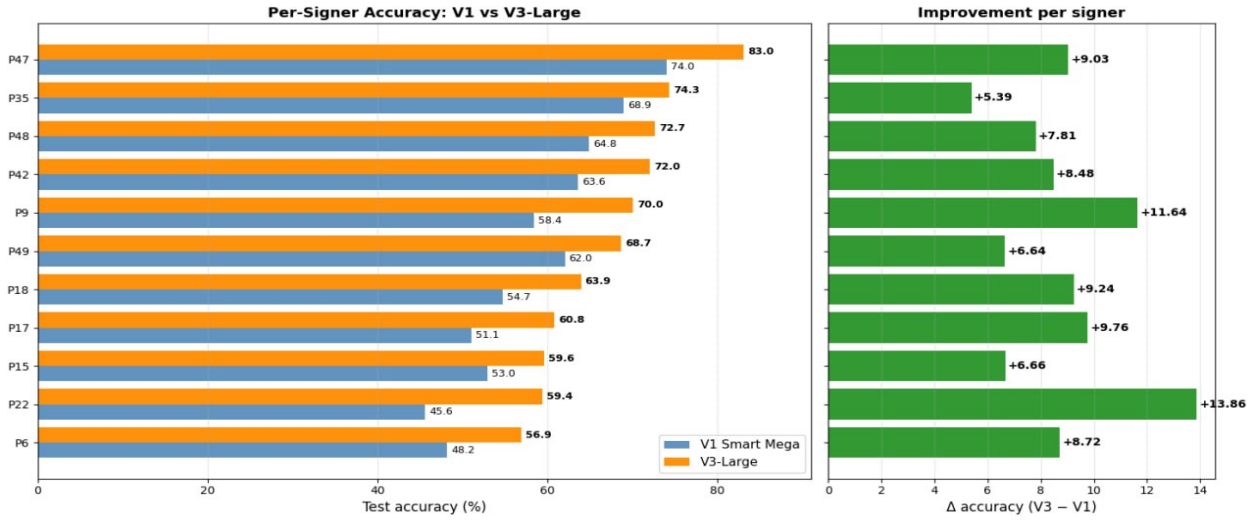

Figure 14 per-sign accuracy

The best performance was achieved on participant **P47**, with **83.04%** accuracy. The lowest performance was observed on participant **P6**, with **56.90%** accuracy. This variation shows that signer differences remain one of the major challenges in ASL recognition.
However, the proposed model improved all 11 test signers compared with the previous baseline. This indicates that the improved architecture and training strategy increased the model's robustness across different signers.

6.4 Results Comparison

This section compares the proposed model with the previous baseline model and discusses the improvement achieved by the V3-Large architecture. The comparison focuses on test Top-1 accuracy because the test set provides the most important evaluation of generalization.

6.4.1 Comparison Between Proposed Models

The previous baseline model achieved **58.56%** test Top-1 accuracy. The proposed **SignTransformer V3-Large** model achieved **67.40%** test Top-1 accuracy. This represents an improvement of: **67.40−58.56=8.84**
Therefore, the proposed model improved test Top-1 accuracy by **8.84 percentage points**.

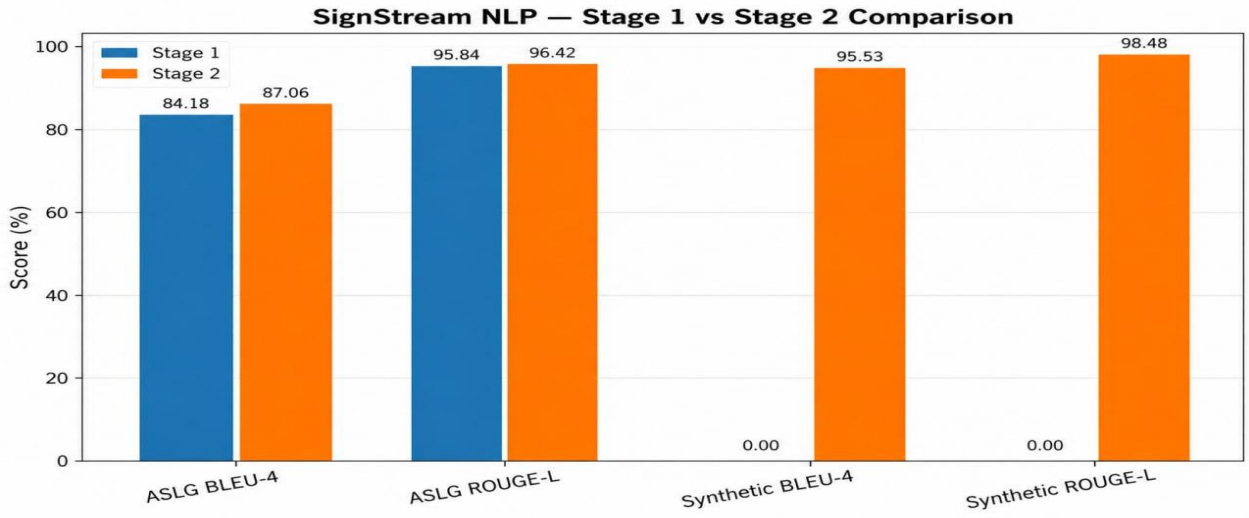

Figure 15 Results Comparison

The improvement can be explained by several changes in the proposed model and training strategy:

1. The model dimension was increased to 384.
2. The Transformer depth was increased to 8 layers.
3. The number of attention heads was increased to 12.
4. Temporal CNN channels were increased to [96,192,384]
5. GELU activation was used instead of simpler activations.

6. DropPath and stronger regularization were applied.
7. EMA was used for more stable evaluation.
8. Mixup and stronger data augmentation were applied.
9. Weight decay and label smoothing were increased to improve generalization.

These changes allowed the model to learn stronger temporal representations and reduce overfitting.

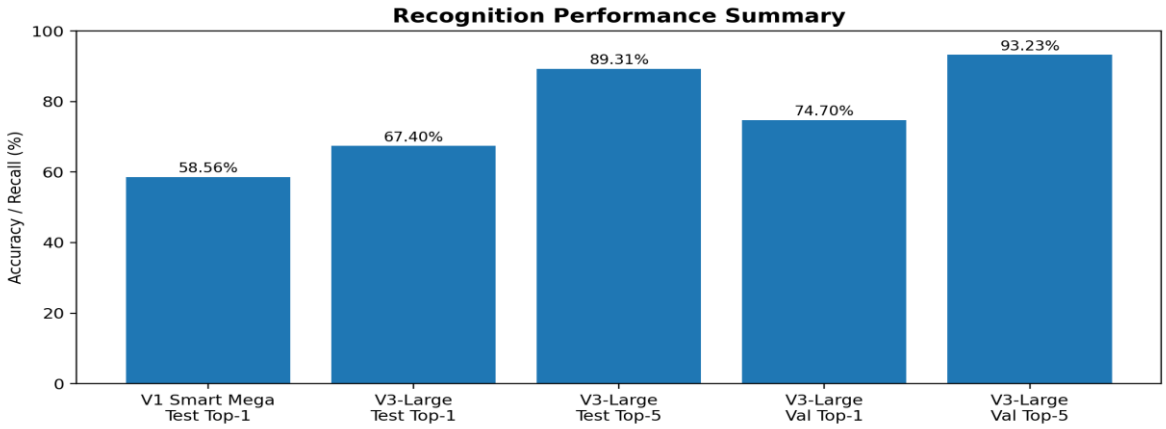

Figure 16 Recogntion Performance Summary

6.4.2 Comparison with Previous Works

Compared with traditional sign language recognition systems, the proposed SignStream system has several advantages. Numerical methods and classical machine learning approaches usually depend on handcrafted features and manually designed rules. These methods may work for small controlled datasets, but they are difficult to scale to thousands of sign classes.
The proposed system uses a keypoint-based deep learning approach. MediaPipe landmarks reduce background noise and focus the model on body motion. Temporal CNN layers capture local movement patterns, while Transformer encoder layers capture long-range temporal dependencies. This makes the system more suitable for large-vocabulary ASL recognition.
The proposed system is not limited to isolated classification output. It also includes gloss buffering, NLP-based text generation, Text-to-Speech output, and a desktop interface. This makes it closer to a complete assistive communication prototype than a standalone classifier.
However, the comparison also shows that there are remaining limitations. The system still faces challenges with visually similar signs, signer variation, and continuous sentence-level recognition. Future work can improve these aspects by using more advanced language models, continuous sign segmentation, larger training data, and real-time optimization.

6.5 User Interface and Runtime Behavior

The final interface demonstrates the complete pipeline. It includes a live camera tab, an uploaded video tab, and a history/session log. The live camera screen shows the video stream with optional keypoint overlay, current sign prediction, confidence progress, top-5 predictions, inference latency,

effective FPS, accepted words, translation output, speak button, export session button, and clear button.
A screenshot of the demo shows a live camera stream with keypoints overlaid on the upper body and face. The interface displayed an inference latency of approximately 28 ms and an effective FPS of approximately 16.4 in that test. These numbers should be interpreted as a demo snapshot rather than a universal benchmark, because runtime depends on hardware, camera resolution, active model, and system loadThe interface uses PyQt threads so that recognition, video processing, NLP translation, and TTS do not freeze the main UI. This is a key engineering feature because model inference can be slower than normal GUI event handling. Separating workers improves responsiveness and makes the application more practical for demonstration.

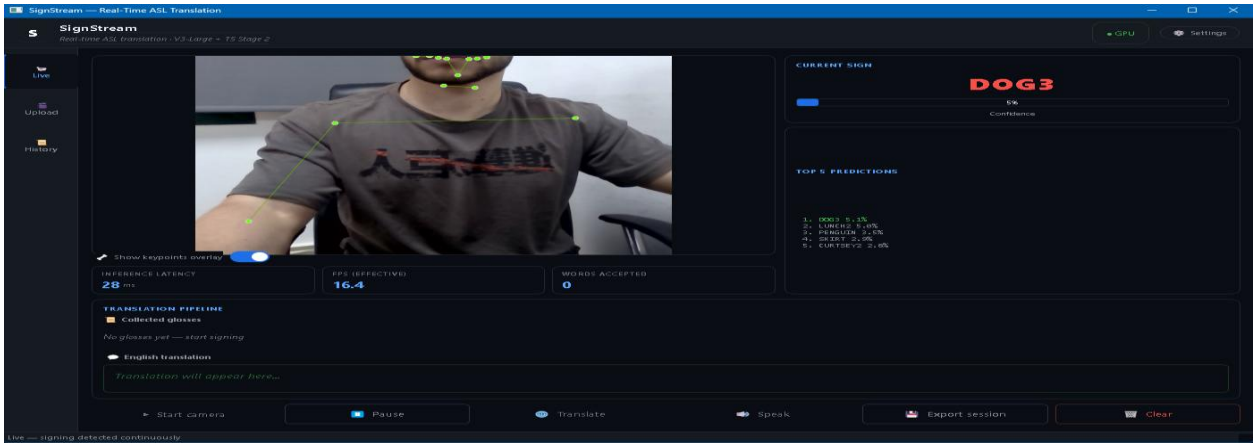

Figure 17 user interface

6.6 Chapter Summary

This chapter presented the experimental evaluation of the SignStream system. The proposed **SignTransformer V3-Large** model achieved **74.70% validation Top-1 accuracy**, **93.23% validation Top-5 accuracy**, **67.40% test Top-1 accuracy**, and **89.31% test Top-5 accuracy**. These results show that the proposed model can recognize ASL glosses from keypoint sequences with strong performance on a large-vocabulary dataset.
The model improved the previous baseline test Top-1 accuracy from **58.56%** to **67.40%**, achieving an improvement of **8.84 percentage points**. The per-signer analysis showed that all test signers improved, which indicates better generalization across different participants.
The results confirm that combining keypoint extraction, temporal CNN layers, Transformer encoders, strong regularization, and data augmentation improves ASL recognition performance.
The chapter also showed that the system is not only a classification model, but part of a complete pipeline that includes gloss buffering, NLP-based text generation, Text-to-Speech, and a user interface.
The next chapter concludes the thesis by summarizing the main findings, discussing limitations, and presenting future work.

Chapter 7

Gaps and Future Work

7.1 Challenges and Future Work

Although the proposed SignStream system achieved promising results, several challenges remain.

One major challenge is **signer variation**. Different people may perform the same ASL sign with different speeds, hand positions, body postures, and movement styles. Although the proposed model improved performance across all test signers, the per-signer results still showed performance differences. This means that signer-independent ASL recognition remains a difficult problem.

Another challenge is the presence of **visually similar signs**. Some ASL glosses share similar hand shapes, movement directions, or body locations. In such cases, the model may confuse one gloss with another. This explains the gap between Top-1 and Top-5 accuracy, where the correct sign may appear among the top predictions but not always as the first prediction.

A third challenge is **continuous sign language recognition**. The dataset used in this project is mainly based on isolated signs, while real communication usually contains continuous signing with multiple signs in one sentence. This requires accurate sign segmentation, start/end detection, word buffering, and sentence-level modeling.

The system may also be affected by **keypoint extraction errors**. MediaPipe may fail to detect hands or facial landmarks in cases of fast motion, occlusion, motion blur, poor lighting, or unusual camera angles. Since the recognition model depends on extracted keypoints, errors in the feature extraction stage can affect the final prediction.

The NLP and TTS components also provide important future development opportunities. In the current system, NLP is used as a post-processing stage to improve gloss-to-text output, and TTS is used to generate speech from the final text. Future versions can improve this stage by fine-tuning a dedicated language model on gloss-to-English sentence pairs. This would make the generated sentences more natural and more context-aware.

Future work can focus on the following improvements:

1. **Continuous Sign Segmentation:**
   Improve the system so it can detect the beginning and end of each sign automatically in live video.
2. **Better Gloss-to-Text Translation:**
   Fine-tune a stronger NLP model to convert ASL gloss sequences into more natural English sentences.
3. **Confidence-Based Unknown Detection:**
   Add a mechanism that rejects uncertain predictions instead of forcing the model to always choose a gloss.
4. **Real-Time Optimization:**
   Optimize the model for faster inference using quantization, pruning, or lighter Transformer variants.

5. **Larger and More Diverse Training Data:**
   Add more signers, lighting conditions, camera angles, and real-world environments to improve generalization.
6. **Improved Keypoint Robustness:**
   Combine MediaPipe with additional tracking or smoothing methods to reduce missing landmarks and noisy detections.
7. **Sentence-Level Evaluation:**
   Evaluate the full system not only at the gloss classification level, but also at the sentence output and speech output levels.
8. **Mobile or Web Deployment:**
   Convert the desktop prototype into a mobile or web-based application to make it easier to use in real-world scenarios.

In conclusion, SignStream demonstrates that combining computer vision, keypoint extraction, temporal deep learning, NLP, TTS, and user interface design can produce a practical ASL recognition and translation prototype. While the current system achieves strong results for isolated sign recognition, future improvements can move it closer to full real-time continuous sign language translation.

Chapter 8

Conclusion

8.1 Conclusions

This thesis presented **SignStream: Real-Time ASL Recognition and Translation System**, an assistive artificial intelligence system designed to recognize American Sign Language signs from video input and convert the recognized output into text and speech. The project addressed an important communication problem faced by deaf and hard-of-hearing individuals, especially when communicating with people who do not understand sign language.

The proposed system followed a complete pipeline that starts from video input and ends with spoken output. The system accepts either live webcam input or uploaded video input. It then extracts keypoint-based features from the hands, upper-body pose, and selected facial regions using MediaPipe. These features are converted into temporal sequences and passed to the proposed visual recognition model, **SignTransformer V3-Large**, which predicts the corresponding ASL gloss.

The proposed model was trained and evaluated on the ASL Citizen dataset using **2,731 ASL gloss classes**. Each sample was represented using a sequence of **60 frames**, with **1725 features per frame**. The final processed dataset contained **83,387 samples**, divided into training, validation, and test sets.

The experimental results showed that the proposed model achieved strong performance for large-vocabulary ASL recognition. The model reached **74.70% validation Top-1 accuracy** and **93.23% validation Top-5 accuracy**. On the test set, the model achieved **67.40% Top-1 accuracy** and **89.31% Top-5 accuracy** using EMA evaluation. Compared with the previous baseline model, which achieved **58.56%** test Top-1 accuracy, the proposed model improved test performance by **8.84 percentage points**.

These results confirm that keypoint-based temporal modeling is effective for ASL recognition. The combination of temporal CNN layers and Transformer encoder layers allowed the model to learn both short-term motion patterns and long-range temporal relationships. The use of augmentation, label smoothing, mixup, EMA, and stronger regularization also helped improve generalization.

In addition to the visual recognition model, the project integrated **NLP-based gloss-to-text processing** and a **Text-to-Speech module**. The NLP stage improves the readability of the recognized gloss sequence, while the TTS stage converts the final generated text into spoken audio. This makes SignStream more than a sign classification model; it becomes a complete prototype that converts sign language video into both text and speech.

The developed PyQt5 user interface also supports practical interaction with the system. It provides live camera mode, uploaded video mode, prediction display, Top-5 results, confidence scores, accepted glosses, generated text, speech output, and session history. Therefore, the project successfully demonstrates an integrated ASL recognition and translation system suitable for academic evaluation and future development.

References

[1] A. Desai et al. (2023), **ASL Citizen: A Community-Sourced Dataset for Advancing Isolated Sign Language Recognition**, Advances in Neural Information Processing Systems, Datasets and Benchmarks. (arxiv.org)

[2] Microsoft Research (n.d.), **ASL Citizen**, project page. (microsoft.com)

[3] Google AI Edge (n.d.), **Holistic landmarks detection task guide**, MediaPipe documentation. (ai.google.dev)

[4] Google Research Blog (2020), **MediaPipe Holistic - Simultaneous Face, Hand and Pose Prediction, on Device**, Google Research Blog. (research.google)

[5] A. Vaswani et al. (2017), **Attention Is All You Need**, Advances in Neural Information Processing Systems. (arxiv.org)

[6] PyTorch Documentation (n.d.), **TransformerEncoder**, PyTorch API documentation. (pytorch.org)

[7] C. Raffel et al. (2020), **Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer**, Journal of Machine Learning Research. (arxiv.org)

[8] N. C. Camgoz, O. Koller, S. Hadfield, and R. Bowden (2020), **Sign Language Transformers: Joint End-to-end Sign Language Recognition and Translation**, CVPR. (arxiv.org)

[9] N. C. Camgoz, S. Hadfield, O. Koller, H. Ney, and R. Bowden (2018), **Neural Sign Language Translation**, CVPR. (openaccess.thecvf.com)

[10] A. Othman and M. Jemni (2012), **English-ASL Gloss Parallel Corpus 2012: ASLG-PC12**, LREC. (achrafothman.net)

[11] A. A. Alkhoraif et al. (2025), **Ensemble Transformer-Based Word-Level Sign Language Recognition with Multi-Modal Input Fusion**, Journal of King Saud University - Computer and Information Sciences. (sciencedirect.com)

[12] K. Roh, H. Lee, E. J. Hwang, S. Cho, and J. C. Park (2024), **Preprocessing Mediapipe Keypoints with Keypoint Reconstruction and Anchors for Isolated Sign Language Recognition**, Proceedings of the LREC-COLING 2024 11th Workshop on the Representation and Processing of Sign Languages. (aclanthology.org)

[13] M. Pu, M. K. Lim, and C. Y. Chong (2025), **Siformer: Feature-isolated Transformer for Efficient Skeleton-based Sign Language Recognition**, arXiv preprint. (arxiv.org)

[14] Y. Min et al. (2025), **A Closer Look at Skeleton-based Continuous Sign Language Recognition**, ICCVW. (openaccess.thecvf.com)

[15] R. Zuo, F. Wei, and B. Mak (2023), **Natural Language-Assisted Sign Language Recognition**, CVPR. (openaccess.thecvf.com)

[16] L. Hu, L. Gao, Z. Liu, and W. Feng (2023), **Continuous Sign Language Recognition with Correlation Network**, CVPR. (openaccess.thecvf.com)

[17] R. Zuo, F. Wei, and B. Mak (2024), **Towards Online Continuous Sign Language Recognition and Translation**, Proceedings of EMNLP. (aclanthology.org)

[18] Y. Hamidullah, J. van Genabith, and C. Espana-Bonet (2024), **Sign Language Translation with Sentence Embedding Supervision**, Proceedings of ACL, Short Papers. (aclanthology.org)

[19] R. Wong, N. C. Camgoz, and R. Bowden (2024), **Sign2GPT: Leveraging Large Language Models for Gloss-Free Sign Language Translation**, ICLR. (openreview.net)